

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

UNIHUB

AUTOR: ALEJANDRO RAMOS RODRÍGUEZ

Puerto Real, Cádiz – 2026

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

UNIHUB

AUTOR: ALEJANDRO RAMOS RODRÍGUEZ

Puerto Real, Cádiz – 2026

Declaración personal de autoría

Alejandro Ramos Rodríguez, estudiante del Grado en Ingeniería Informática en la Escuela Superior de Ingeniería de la Universidad de Cádiz, como autor de este documento académico titulado *UniHub* y presentado como Trabajo Final de Grado.

DECLARO QUE

Es un trabajo original, que no copio ni utilizo parte de obra alguna sin mencionar de forma clara y precisa su origen tanto en el cuerpo del texto como en su bibliografía y que no empleo datos de terceros sin la debida autorización, de acuerdo con la legislación vigente. Asimismo, declaro que soy plenamente consciente de que no respetar esta obligación podrá implicar la aplicación de sanciones académicas, sin perjuicio de otras actuaciones que pudieran iniciarse.

En Puerto Real, a 28 de agosto de 2026

Fdo.: Alejandro Ramos Rodríguez

Agradecimientos

Deseo expresar mi más sincero agradecimiento a la Escuela Superior de Ingeniería (ESI) y al cuerpo docente de la Universidad de Cádiz (UCA) por la formación recibida a lo largo de los estudios del Grado en Ingeniería Informática.

A mi familia y compañeros por su apoyo incondicional durante todo el proceso de desarrollo e investigación del presente Trabajo Fin de Grado.

Resumen

El presente Trabajo Fin de Grado aborda el diseño, desarrollo e implementación de **UniHub**, un sistema informático distribuido e integral para la centralización, almacenamiento, consulta y gestión de la oferta de educación superior en España (universidades públicas y privadas, titulaciones de Grado, Máster y Doctorado, y planes de estudio desglosados del Boletín Oficial del Estado - BOE).

El proyecto se estructura en cuatro fases de ingeniería perfectamente delimitadas e interconectadas:

1. **Fase 1 (Rastreador / Crawler):** Desarrollado en Python, obtiene el catálogo oficial, filtra titulaciones vigentes y analiza las resoluciones del BOE. El análisis de PDF construye un modelo de páginas, posiciones y tablas para delimitar cada titulación y extraer su plan curricular. La ejecución registra su trazabilidad, respeta `robots.txt` en las fuentes web y admite una planificación Cron configurable mediante `CRAWLER_CRON_SCHEDULE`.
2. **Fase 2 (Base de Datos PostgreSQL y API REST):** Modelo relacional DDL con rol de solo lectura de API (`unihub_api_user`), pipeline de carga ETL, ordenación prioritaria (públicas primero, luego privadas), soporte de métricas físicas individuales de contenedores cgroup y servicio REST en FastAPI.
3. **Fase 3 (Portal Web Frontend):** Aplicación Web SPA moderna creada con React y Vite bajo la identidad visual de la Universidad de Cádiz (UCA). Incluye paginación configurable (5, 10, 20, 50 y 100 por página), ocultación de códigos internos, geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual `/admin`.
4. **Fase 4 (Contenerización y Orquestación Docker):** Sistema de 4 contenedores aislados (`unihub_db`, `unihub_crawler`, `unihub_api`, `unihub_www`) orquestados con Docker Compose con garantía de persistencia permanente de datos tras apagar o reiniciar los servicios.

Palabras clave: UniHub, Universidad de Cádiz (UCA), Rastreador Python, BOE, Doctorado, PostgreSQL, FastAPI, React, Geolocalización Haversine, Docker Compose.

Índice general

Índice de figuras	x
-------------------	---

Índice de tablas	xi
------------------	----

I Prolegómeno	1
----------------------	----------

1. Introducción	2
------------------------	----------

1.1. Motivación	2
---------------------------	---

1.2. Alcance y Objetivos	2
------------------------------------	---

1.3. Glosario de términos	3
-------------------------------------	---

1.4. Organización del documento	3
---	---

2. Antecedentes	5
------------------------	----------

2.1. Contexto	5
-------------------------	---

2.2. Estado de la técnica	5
-------------------------------------	---

3. Plan de gestión de proyecto	7
---------------------------------------	----------

3.1. Metodología de desarrollo	7
--	---

3.2. Tecnologías	7
----------------------------	---

3.3. Herramientas utilizadas	7
--	---

3.4. Planificación del proyecto	8
---	---

3.5. Organización	8
-----------------------------	---

3.6. Costes	9
-----------------------	---

3.7. Riesgos	9
------------------------	---

3.8. Gestión de la configuración	9
--	---

3.9. Aseguramiento de calidad	9
---	---

II Desarrollo	10
----------------------	-----------

4. Requisitos del Sistema	11
----------------------------------	-----------

4.1. Objetivos de Negocio	11
-------------------------------------	----

4.2. Objetivos del Sistema	11
--------------------------------------	----

4.3. Requisitos funcionales	11
---------------------------------------	----

4.4. Requisitos no funcionales	13
--	----

4.5. Requisitos de información	13
--	----

4.6. Reglas de negocio	13
----------------------------------	----

4.7. Análisis GAP	14
-----------------------------	----

5. Análisis del Sistema	17
--------------------------------	-----------

5.1. Modelo Conceptual y de Dominio	17
---	----

5.2. Modelo de Casos de Uso	17
---------------------------------------	----

5.3.	Modelo de Comportamiento Dinámico	19
5.3.1.	Modelo de Comportamiento del Rastreador y Validación Curricular (Fase 1)	19
5.3.2.	Modelo de Secuencia: Simulación Financiera en Calculadora de Matrícula	20
5.4.	Modelo de Interfaz de Usuario	20
6.	Diseño del Sistema	23
6.1.	Arquitectura del Sistema	23
6.1.1.	Diseño de alto nivel	23
6.1.2.	Diseño detallado	23
6.1.3.	Seguridad y Control de Acceso (RBAC)	25
6.2.	Parametrización del software base	25
6.3.	Diseño Físico de Datos	25
6.4.	Diseño de la Interfaz de Usuario	26
7.	Construcción del Sistema	29
7.1.	Entorno de Construcción	29
7.2.	Código Fuente y Estructura Modular	29
7.2.1.	Fase 1: Módulo de Ingesta y Crawler (Codigo/Crawler/)	29
7.2.2.	Fase 2: Módulo de API REST y Persistencia (Codigo/API/)	38
7.2.3.	Fase 3: Módulo de Interfaz Web y Calculadora (Codigo/WWW/)	38
7.2.4.	Fase 4: Despliegue Contenerizado y Orquestación (Codigo/Docker/)	39
7.2.5.	Pruebas y Auditoría de Rendimiento (Codigo/Pruebas/)	40
7.3.	Código DDL de Base de datos	40
8.	Pruebas del Sistema	45
8.1.	Estrategia de Pruebas	45
8.2.	Pruebas Unitarias y de Integración	45
8.3.	Validación del Parser de Resoluciones BOE	46
8.3.1.	Evaluación Empírica Reproducible	46
8.4.	Pruebas de Rendimiento y Benchmarking	47
8.5.	Pruebas de Ejecución Controlada	47
8.6.	Pruebas No Funcionales	47
8.7.	Criterio de Aceptación	48
9.	Despliegue del Sistema	51
9.1.	Arquitectura Física	51
9.2.	Instrucciones de despliegue	51
9.2.1.	Requisitos previos	51
9.2.2.	Inventario de componentes	51
9.2.3.	Procedimientos de instalación	52
9.2.4.	Pruebas de implantación	53
9.3.	Instrucciones para la operación del sistema y mantenimiento del nivel de servicio	53

III	Epílogo	54
10.	Conclusiones	55
10.1.	Objetivos alcanzados	55
10.1.1.	Resultados Cuantitativos y Métricas Clave	55
10.2.	Lecciones aprendidas	56
10.3.	Trabajo futuro	56
	Información sobre Licencia	59
	Bibliografía	61
IV	Anexos	63
A.	Manual del desarrollador	64
A.1.	Introducción	64
A.2.	Preparación del entorno de trabajo	64
A.3.	Consideraciones generales sobre el desarrollo	64
A.4.	Instrucciones para construcción	65
A.5.	Ejecución Avanzada y Filtrado del Crawler (Fase 1)	65
B.	Manual de usuario	69
B.1.	Introducción	69
B.2.	Instalación	69
B.3.	Uso del sistema	69
C.	Glosario de Términos y Acrónimos Técnicos	73
C.1.	Glosario de Conceptos y Patrones de Diseño	73

Índice de figuras

5.1.	Diagrama de Clases del Modelo de Dominio de UniHub (UML)	18
5.2.	Diagrama General de Casos de Uso de UniHub (UML)	18
5.3.	Diagrama de Actividad: Ingesta Inteligente y Validación Matemática de Completitud Curricular	19
7.1.	Diagrama de arquitectura paralela Productor-Consumidor, caché SHA-256 y resiliencia Circuit Breaker de la Fase 1 Parte 1	32
7.2.	Flujo de rastreo web oficial con arquitectura Hub-and-Spoke, verificación robots.txt, sitemaps XML y extracción tarifaria privada de la Fase 1 Parte 2	34
7.3.	Algoritmo de cómputo tarifario a partir del catálogo local, cacheo en memoria RAM y sincronización de la Fase 1 Parte 3	35
7.4.	Diagrama de arquitectura paralela Productor-Consumidor y resiliencia de red de la Fase 1	41

Índice de tablas

Parte I

Prolegómeno

1. Introducción

A continuación, se describe la motivación del proyecto **UniHub** y su alcance. También se incluye un glosario de términos y la organización del resto de la presente documentación.

1.1. Motivación

El catálogo de la oferta universitaria de enseñanza superior en España almacena la información de titulaciones oficiales. Sin embargo, la consulta integrada de universidades públicas y privadas, la verificación de planes de estudio vigentes modificados en el Boletín Oficial del Estado (BOE) y la clasificación de Grados, Másteres y Doctorados resultaba compleja y fragmentada.

La motivación principal de este proyecto es construir **UniHub**, un sistema informático moderno, robusto y automatizado que rastree, homogeneice, persista y exponga de forma interactiva la oferta universitaria oficial de España, facilitando la visualización de asignaturas, módulos y créditos ECTS cuando la fuente publicada proporcione ese nivel de detalle.

1.2. Alcance y Objetivos

El alcance del proyecto abarca cuatro fases de ingeniería de software independientes pero integradas:

1. **Desarrollo de un Rastreador (Crawler) en Python (Fase 1):** Capaz de obtener los catálogos oficiales, aplicar filtrado estricto de titulaciones vigentes y autorizadas (descartando extinguidas o no vigentes), clasificar Doctorados, inspeccionar todos los candidatos a PDF del BOE, aplicar resiliencia ante fallos de conexión (pausas de 5 min y cortocircuito a los 15 min), registrar métricas y notificar a la API REST tras completar la recolección.
2. **Diseño de Base de Datos y API REST en Python (Fase 2):** Creación de un esquema relacional en PostgreSQL con control de acceso por roles (`ruct_api_user` de solo lectura), pipeline de ingesta ETL, ordenación prioritaria (públicas primero, luego privadas) y construcción de una API REST en FastAPI con métricas físicas de contenedores cgroup.
3. **Desarrollo del Portal Web Interactivo (Fase 3):** Aplicación SPA en Vite + React con el sistema de diseño visual de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100 elementos por página), geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG

Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual /admin.

4. **Sistemas de Contenerización y Orquestación Docker (Fase 4):** Despliegue independiente de 4 contenedores (`ruct_db`, `ruct_crawler`, `ruct_api`, `ruct_www`) en una red privada virtualizada con persistencia garantizada de datos.

1.3. Glosario de términos

UniHub: Nombre oficial de la plataforma web y sistema distribuido objeto de este Trabajo Fin de Grado.

RUCT: Registro oficial de origen de datos de Universidades, Centros y Títulos.

BOE: Boletín Oficial del Estado.

ECTS: European Credit Transfer and Accumulation System (Sistema Europeo de Transferencia y Acumulación de Créditos).

API REST: Representational State Transfer Application Programming Interface.

SPA: Single Page Application (Aplicación de Página Única).

UCA: Universidad de Cádiz.

ETL: Extract, Transform and Load (Extracción, Transformación y Carga).

Web Vitals: Métricas de rendimiento de la experiencia de usuario en la web (FCP, LCP, TTFB).

Haversine: Fórmula matemática para calcular distancias entre dos puntos en una esfera a partir de sus coordenadas geográficas.

1.4. Organización del documento

La presente memoria se estructura en las siguientes secciones principales:

- **Prolegómeno:** Introducción, antecedentes del dominio universitario y plan de gestión de proyecto.
- **Desarrollo:** Catálogo de requisitos, análisis UML, diseño arquitectónico (patrón C4/capas), implementación en código de cada fase, pruebas ejecutadas y despliegue del sistema Docker.
- **Epílogo:** Conclusiones generales, lecciones aprendidas y líneas de trabajo futuro.
- **Anexos:** Manuales de usuario, guía de desarrollador y ejemplos de tablas e imágenes.

2. Antecedentes

En este capítulo se detalla la situación actual que origina el desarrollo del sistema **UniHub**. Asimismo, se describen las alternativas tecnológicas existentes.

2.1. Contexto

El panorama de la educación superior en España está integrado por más de un centenar de universidades públicas y privadas, distribuidas en las 17 comunidades autónomas. Cada año se publican y modifican miles de titulaciones oficiales de Grado, Máster y Doctorado en el Boletín Oficial del Estado (BOE).

Las fuentes de consulta oficiales proporcionan la información en listados tabulados XLS o páginas en HTML. Sin embargo, carecen de una interfaz moderna de búsqueda unificada por geolocalización, no desglosan visualmente las asignaturas y créditos ECTS extraídos de las publicaciones BOE, no permiten filtrar Doctorados de forma diferenciada ni ordenan prioritariamente los centros públicos frente a los privados. La problemática radica en la dispersión de los datos y la falta de un sistema integrado de consulta pública y administración contenerizada.

2.2. Estado de la técnica

Existen plataformas estatales e institucionales de consulta académica, entre las que destacan:

- **Registro Oficial de Origen (Ministerio de Educación):** Fuente primaria de los datos. Proporciona búsquedas por formulario básico y exportación en formato legacy BIFF8 (.xls). No incluye servicios API REST modernos ni interfaz orientada a la experiencia de usuario móvil/geolocalizada.
- **Boletín Oficial del Estado (BOE):** Repositorio jurídico oficial donde se publican las resoluciones y planes de estudio en formato PDF. Requiere herramientas especializadas de procesamiento de lenguaje natural y parseo de tablas para extraer asignaturas y módulos estructurados.
- **Portales Web Propietarios de Universidades:** Cada universidad publica sus planes de estudio con formatos visuales dispares, dificultando la comparativa directa entre titulaciones equivalentes.

El sistema **UniHub** aborda esta brecha mediante un pipeline automatizado de cuatro componentes: rastreo, almacenamiento relacional en PostgreSQL, exposición mediante API REST FastAPI y visualización interactiva en React bajo el sistema de diseño visual de la Universidad de Cádiz (UCA).

3. Plan de gestión de proyecto

En esta sección se describen todos los aspectos relativos a la gestión del proyecto **UniHub**: metodología, organización, costes, planificación, riesgos y aseguramiento de la calidad.

3.1. Metodología de desarrollo

Se ha adoptado una metodología de desarrollo ágil e incremental estructurada en 4 fases principales. Cada fase se concibe como un subsistema desacoplado con responsabilidades claras, validado mediante pruebas unitarias e integrales antes de avanzar a la siguiente iteración.

3.2. Tecnologías

Las tecnologías empleadas se resumen a continuación:

- **Fase 1 (Crawler):** Python 3.12, `xlrd` (lectura de XLS BIFF8), `BeautifulSoup4` (análisis HTML), `pdfplumber` (modelo geométrico y tablas del PDF BOE), OCR opcional para documentos escaneados, `psutil` (medición de memoria/CPU) y ejecución Cron configurable.
- **Fase 2 (API & DB):** PostgreSQL 15, SQLAlchemy 2.0 (ORM), FastAPI (framework REST), Pydantic v2 (validación de datos) y `psycpg2-binary`.
- **Fase 3 (Web):** Node.js 20, Vite, React 19 (JSX), Vanilla CSS (Tokens UCA), Lucide Icons, Geolocation API y analítica local `usageTracker`.
- **Fase 4 (Docker):** Docker Engine, Docker Compose v2, Nginx 1.25-alpine e imágenes base slim de Python 3.12.

3.3. Herramientas utilizadas

Para el desarrollo y la documentación del proyecto se emplearon las siguientes herramientas de soporte:

- **Entorno de desarrollo:** Visual Studio Code, Git, MiKTeX para compilación de LaTeX.
- **Asistencia a la redacción y revisión:** Google Antigravity con Gemini 3.6 Flash para la revisión de estilo académico y coherencia terminológica.

- **Sistema de asistencia a la ingeniería:** Sistema montado por Unsloth Studio + Muse-Glimmer-30B-GGUF + OpenCode para la exploración del código, detección de inconsistencias entre la implementación y la documentación y generación de anexos técnicos.

3.4. Planificación del proyecto

El desarrollo del proyecto se ejecutó en 4 hitos secuenciales:

1. **Hito 1 - Rastreador (Fase 1):** Implementación de descargas con reintentos, deduplicación de planes renovados por Real Decreto, clasificación de Doctorados, filtro estricto de vigencia (descarte de extinguidas), resiliencia de conexión (5m/15m cortocircuito), parseo de PDF del BOE y notificación automática a la API REST.
2. **Hito 2 - Base de Datos y API REST (Fase 2):** Diseño del esquema DDL en PostgreSQL, rol de solo lectura (`ruct_api_user`), script ETL, ordenación prioritaria (públicas primero) y endpoints de consulta y métricas físicas cgroup.
3. **Hito 3 - Portal Web UCA & CRUD (Fase 3):** Desarrollo de la SPA en React con estética UCA, paginación (5, 10, 20, 50, 100), ocultación de códigos internos, geolocalización por Haversine con distancia en tarjeta, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual `/admin`.
4. **Hito 4 - Dockerización & Persistencia (Fase 4):** Creación de Dockerfiles, Nginx proxy inverso, `docker-compose.yml` y montaje de volúmenes persistentes (`ruct.postgres.data` y datos en host).

3.5. Organización

El proyecto ha sido coordinado bajo la estructura de roles de un proyecto de ingeniería de software:

- **Jefe de Proyecto / Analista:** Definición de requisitos, arquitectura general y planificación por fases.
- **Desarrollador Backend / Crawler:** Construcción del rastreador Python, modelos relacionales y API REST FastAPI.
- **Desarrollador Frontend:** Implementación de la SPA en React, sistema de diseño UCA y motores de métricas.
- **Ingeniero de DevOps:** Configuración de contenedores Docker, proxies Nginx y persistencia de volúmenes.

3.6. Costes

Se estiman los costes de recursos humanos y equipamiento:

- **Personal (Ingeniería de Software):** 350 horas de trabajo estimadas a una tarifa media de 30 €/h = 10.500 €.
- **Equipamiento y Software:** Uso de herramientas y tecnologías de Software Libre / Código Abierto (Python, PostgreSQL, FastAPI, React, Nginx, Docker), con un coste de licencias de 0 €.
- **Coste Total Estimado:** 10.500 €.

3.7. Riesgos

Se identificaron y mitigaron los siguientes riesgos principales:

- **R-01: Cambios en el formato de exportación o cortes de red.** Impacto: Alto. Mitigación: Parsers flexibles con manejo de excepciones, log de errores en `errores_crawler.json` y cortocircuito con pausas de 5m y 15m.
- **R-02: Sobrescritura accidental de datos válidos con valores vacíos.** Impacto: Alto. Mitigación: Implementación de la función estricta de validación `is_valid_value` en el crawler.
- **R-03: Pérdida de datos al apagar contenedores Docker.** Impacto: Crítico. Mitigación: Mapeo de volúmenes nominados en PostgreSQL y montajes directos en el disco host (`../Crawler/Datos`).

3.8. Gestión de la configuración

Control de versiones gestionado mediante Git en el repositorio del proyecto. Se aplicó una política de commits atómicos e independientes para cada tarea funcional del proyecto.

3.9. Aseguramiento de calidad

Se establecieron controles de calidad continuos: compilación del bundle React en Vite con 0 errores (211ms), validación estricta de schemas Pydantic en FastAPI, comprobaciones de salud de la base de datos (`pg_isready`) y verificación de compilación del documento LaTeX en MiKTeX.

Parte II

Desarrollo

4. Requisitos del Sistema

En este capítulo se presentan los objetivos del sistema y el catálogo completo de requisitos funcionales y no funcionales de la plataforma **UniHub**.

4.1. Objetivos de Negocio

El objetivo de negocio principal es ofrecer un portal público centralizado de enseñanza superior en España, facilitando la transparencia, la comparativa de planes de estudio BOE y el acceso a la oferta universitaria pública y privada.

4.2. Objetivos del Sistema

1. Automatizar la extracción periódica de universidades y titulaciones vigentes (Grados, Másteres y Doctorados). 2. Homogenizar y almacenar relacionalmente los datos en PostgreSQL ordenando públicamente primero las universidades públicas y posteriormente las privadas. 3. Exponer una API REST documentada en FastAPI con control de permisos y métricas físicas cgroup. 4. Construir un portal web interactivo con estética corporativa de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100) y ocultación de códigos internos. 5. Proporcionar un buscador por geolocalización (cercanía en km con distancia integrada directamente en tarjetas). 6. Recolectar métricas de uso y clasificar las 6 universidades más visitadas en "Universidades Destacadas". 7. Incorporar el apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA). 8. Aislar el panel de control del Administrador de la Web tras la ruta manual /admin. 9. Desplegar la infraestructura mediante contenedores Docker orquestados con persistencia garantizada.

4.3. Requisitos funcionales

- **RF-01 Descarga de Catálogo Oficial y Computación Paralela:** Descargar e inspeccionar catálogos de universidades y titulaciones mediante una arquitectura en 2 procesos paralelos (Productor de red I/O y Consumidor de análisis CPU).
- **RF-02 Filtrado Estricto de Vigencia y Clasificación de Doctorados:** Excluir titulaciones extinguidas, no vigentes o eliminadas; conservar únicamente las vigentes o autorizadas y clasificar Doctorados (RD 99/2011).
- **RF-03 Parseo Meticuloso Multi-PDF de BOE:** Escanear todos los PDF candidatos del BOE para extraer y fusionar asignaturas, módulos, materias, créditos ECTS, curso y cuatrimestre sin detenerse en la primera coincidencia.

- **RF-04 Resiliencia y Notificación Automática:** Aplicar pausas de 5 min (10 fallos) y cortocircuito a los 15 min ante caídas de red, notificando via POST a la API REST al finalizar.
- **RF-05 Regla de No Sobrescritura Vacía:** Evitar que campos vacíos o indeterminados (, undefined") sobrescriban información previa válida.
- **RF-06 API REST de Consulta con Ordenación Prioritaria:** Endpoints para listar universidades (públicas primero, privadas después) y titulaciones por nivel académico.
- **RF-07 API REST CRUD de Administración y Métricas cgroup:** Endpoints POST, PUT y DELETE para universidades y titulaciones, junto con métricas de uso de RAM/CPU por contenedor.
- **RF-08 Buscador, Filtros y Paginación Web:** Búsqueda global por nombre, tipo (Pública/Privada), CCAA y nivel (Grado, Máster, Doctorado), con paginación de 5, 10, 20, 50 y 100 resultados.
- **RF-09 Ocultación de Códigos Internos:** Ocultar códigos internos numéricos en la interfaz pública web.
- **RF-10 Visor Modal de Planes BOE:** Visualización en la web del desglose ECTS y enlace oficial al PDF del BOE.
- **RF-11 Búsqueda por Cercanía con Distancia Integrada:** Cálculo de distancias en km mediante la fórmula de Haversine mostrando la distancia dentro de la tarjeta del centro.
- **RF-12 Universidades Destacadas (Top 6):** Cálculo dinámico de las 6 universidades más consultadas en la plataforma.
- **RF-13 Sección "Sobre Nosotros":** Página informativa del TFG de Alejandro Ramos Rodríguez (Ingeniería Informática, UCA).
- **RF-14 Panel del Administrador en Ruta Manual /admin:** Acceso aislado sin botón visible para gestionar CRUD y monitorear la salud individual de contenedores.
- **RF-15 Rastreo Paralelo de Webs Oficiales (Fase 1 - Parte 2):** Escanear en paralelo las webs oficiales de las universidades para titulaciones sin plan de estudios, verificando estrictamente robots.txt, analizando el Sitemap XML, buscando mediante 26 sinónimos académicos y guardando la URL directa de acceso en web_fuente_directa_url.
- **RF-16 Validación Matemática de Completitud Curricular:** Validar que el volumen total de créditos ECTS obtenidos en las asignaturas sea $\geq$ a los créditos oficiales exigidos por ley (Grado estándar 240, Medicina 360, Odontología/Farmacia/Veterinaria/Arquitectura 300, Máster 120/90/60, Doctorado RD 99/2011). Si un plan es incompleto o solo resumen, activar automáticamente el rastreo en la web oficial.
- **RF-17 Priorización Contextual Semántica de Enlaces en RUCT:** Clasificar los enlaces a PDFs del BOE según el fieldset contenedor en la ficha del

RUCT, dando máxima prioridad a los enlaces de *Publicación Plan de Estudios* y *Fechas de Corrección Plan Estudio* (prioridad 90–100) y relegando los *Acuerdos de Consejo de Ministros* (prioridad 10).

4.4. Requisitos no funcionales

- **RNF-01 Rendimiento:** Latencia promedio de respuesta de la API REST inferior a 100 ms.
- **RNF-02 Usabilidad e Identidad Visual:** Interfaz web profesional con paleta corporativa de la Universidad de Cádiz (UCA).
- **RNF-03 Seguridad:** Rol de solo lectura en base de datos (`unihub_api_user`) y acceso restringido por ruta manual al panel de administración.
- **RNF-04 Resiliencia a Fallos:** El crawler no se detendrá ante errores individuales de red o parseo de PDFs, registrándolos en `errores_crawler.json` y aplicando el cortocircuito de 5m/15m.
- **RNF-05 Persistencia y Portabilidad:** Despliegue en 4 contenedores Docker independientes con persistencia garantizada en volumen nominado y montaje directo de host.

4.5. Requisitos de información

El sistema gestiona las siguientes entidades principales de información: *Universidad*, *Titulación*, *PlanEstudios*, *ResumenCréditos*, *ElementoCurricular*, *ErrorCrawler* y *EstadísticaRendimiento*.

4.6. Reglas de negocio

- **RN-01 Jerarquía por Real Decreto:** En presencia de múltiples planes de una misma titulación, priorizar RD 822/2021 ¿ RD 1393/2007 ¿ RD 56/2005.
- **RN-02 No Omisión Entre Universidades:** Misma titulación en distintas universidades debe ser tratada como titulación independiente.
- **RN-03 Comprobación Incremental Estricta:** No volver a descargar un PDF del BOE si la URL y la fecha coinciden con el registro de `checkpoint.json`.
- **RN-04 Ordenación Prioritaria:** Mostrar siempre las universidades públicas en primer lugar y posteriormente las privadas.
- **RN-05 Cero Valores por Defecto Arbitrarios:** Prohibir cualquier estimación artificial o fallback estático de tarifas o créditos; todo dato debe provenir estrictamente de las fuentes oficiales (BOE, RUCT, SIIU, web oficial).

4.7. Análisis GAP

Dado que **UniHub** se ha construido a medida mediante una arquitectura en 4 fases que integra rastreo, API REST propia, frontend con diseño UCA y orquestación Docker, no se requirió la adopción directa de plataformas base de terceros.

5. Análisis del Sistema

Este capítulo cubre el análisis conceptual, funcional y dinámico del sistema de información **UniHub**, haciendo uso del lenguaje de modelado UML y modelos de comportamiento formalizados.

5.1. Modelo Conceptual y de Dominio

El modelo conceptual identifica las entidades centrales del dominio universitario y sus asociaciones estructurales:

- **Universidad:** Representa cada centro universitario público o privado registrado en el RUCT (código oficial, nombre, tipo de centro, comunidad autónoma, municipio, provincia, URL web oficial, email de contacto y teléfono institucional).
- **Titulacion:** Representa una titulación oficial vigente de Grado, Máster o Doctorado (código de estudio, denominación oficial, nivel académico, estado de vigencia y rama de conocimiento). Se relaciona mediante agregación con Universidad (1:N).
- **PlanEstudios:** Contiene la metainformación del plan de estudios (URL del BOE, fecha de publicación, fecha de procesado, procedencia de la fuente y diagnóstico de completitud matemática). Se relaciona 1:1 con Titulacion.
- **ResumenCreditos:** Almacena la distribución oficial de créditos ECTS (formación básica, obligatoria, optativa, TFG/TFM, prácticas externas y créditos totales exigidos). Relación N:1 con PlanEstudios.
- **ElementoCurricular:** Almacena el desglose estructurado de asignaturas, módulos, materias, créditos ECTS, carácter (Básica, Obligatoria, Optativa), curso y cuatrimestre. Relación N:1 con PlanEstudios.
- **TarifaECTS:** Almacena la estructura de precios públicos por crédito ECTS (1^a a 4^a matrícula) e importes estimados anuales procedentes del SIIU y decretos autonómicos. Relación N:1 con Titulacion.

5.2. Modelo de Casos de Uso

Se identifican tres actores que interactúan con el sistema según su perfil de permisos:

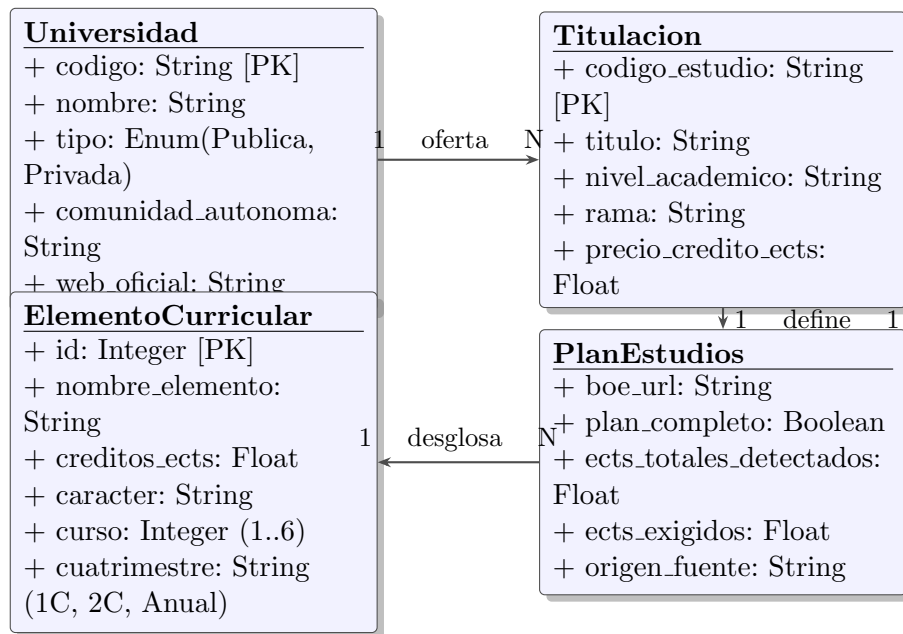


Figura 5.1: Diagrama de Clases del Modelo de Dominio de UniHub (UML)

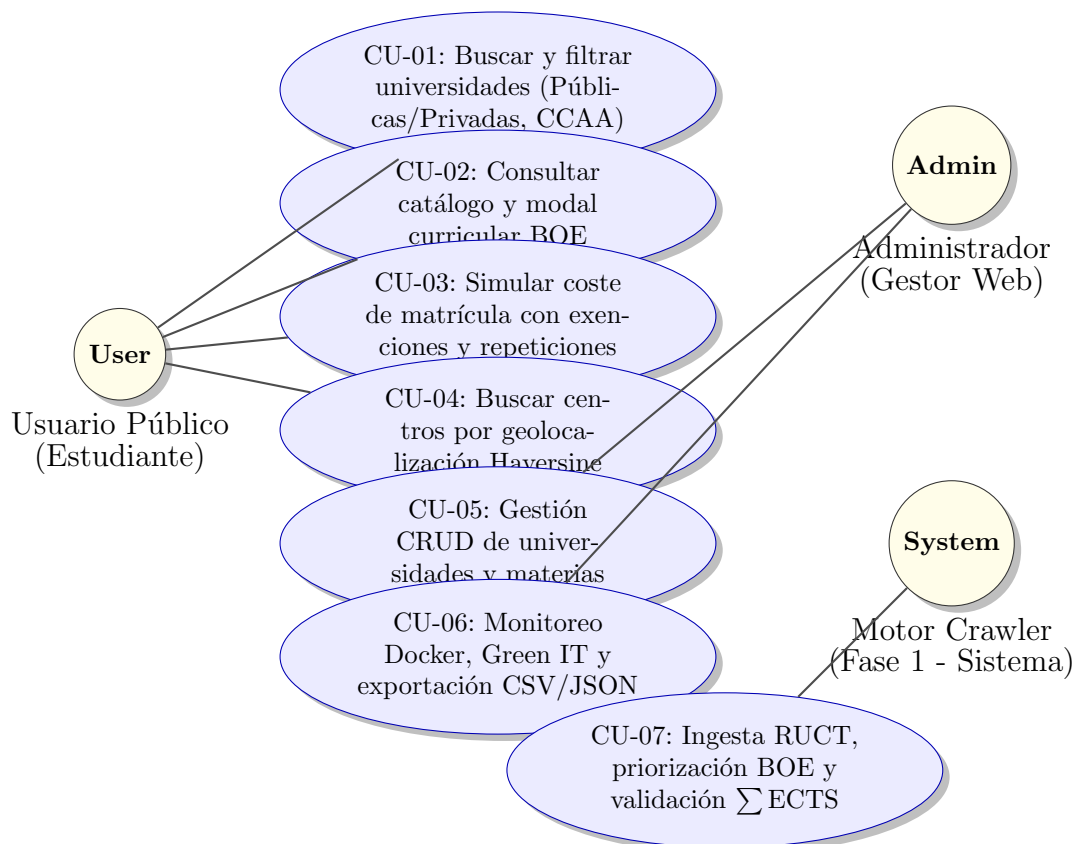


Figura 5.2: Diagrama General de Casos de Uso de UniHub (UML)

5.3. Modelo de Comportamiento Dinámico

El comportamiento del sistema se describe a través de dos modelos de interacción fundamentales:

5.3.1. Modelo de Comportamiento del Rastreador y Validación Curricular (Fase 1)

El flujo de ejecución del rastreador combina la priorización contextual de enlaces HTML en RUCT con la validación matemática de completitud curricular:

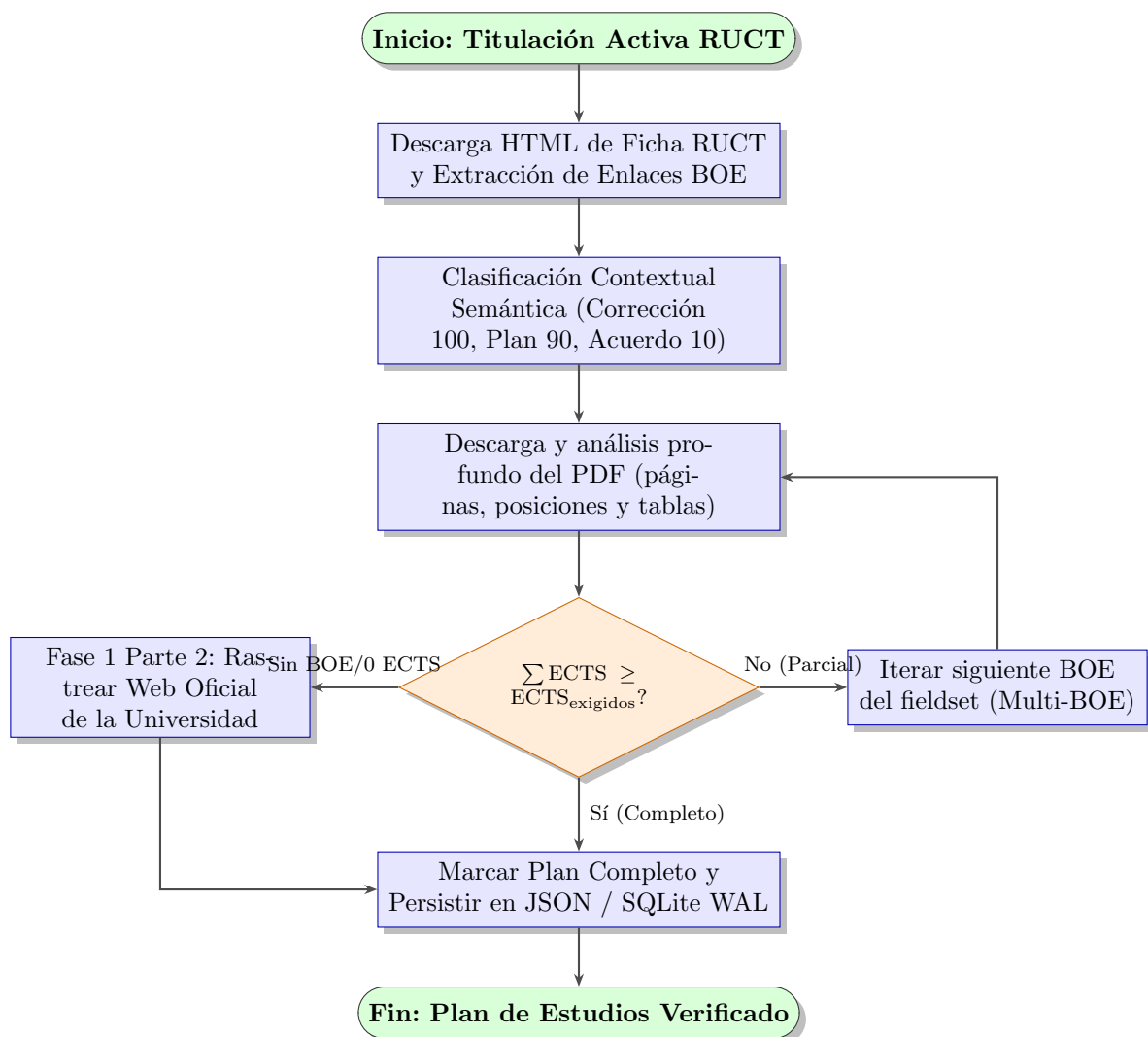


Figura 5.3: Diagrama de Actividad: Ingesta Inteligente y Validación Matemática de Completitud Curricular

5.3.2. Modelo de Secuencia: Simulación Financiera en Calculadora de Matrícula

En la Fase 3, el usuario interactúa dinámicamente con la calculadora de matrícula:

1. El estudiante selecciona una titulación oficial; la aplicación React recupera el desglose de asignaturas del plan de estudios y la tarifa oficial SIIU por crédito ECTS desde la API REST.
2. El usuario marca las asignaturas que desea matricular e indica el número de matrícula (1^a, 2^a, 3^a o 4^a repetición).
3. El motor de cálculo aplica los coeficientes multiplicadores (1,0×, 1,5×, 3,0×, 4,5×), deduce las bonificaciones sociales seleccionadas (Beca MEC, Familia Numerosa, Bonificación 99 % autonómica) y genera el recibo de liquidación en tiempo real (< 1 ms).

5.4. Modelo de Interfaz de Usuario

El diseño de interfaz SPA se estructura en componentes modulares React con soporte nativo de la History API:

- **Barra de Navegación Nivel Superior (Navbar):** Logotipo UniHub, pestañas de navegación (Inicio, Universidades, Titulaciones, Calculadora, Cercanía, Sobre Nosotros) y selector de modo oscuro.
- **Pantalla de Inicio (Hero):** Banner corporativo estilo UCA, métricas globales dinámicas y grilla interactiva de “Universidades Destacadas”.
- **Grilla de Tarjetas con Paginación (UnivCard / DegreeCard):** Vista de fichas con ocultación de códigos internos, distancias en kilómetros integradas y distintivo visual de plan completo verificado.
- **Modal de Plan de Estudios (PlanModal):** Cuadro de diálogo flotante con desenfoque de fondo (*glassmorphism*) que lista la estructura curricular, itinerarios de mención y enlace al PDF oficial del BOE.
- **Panel de Administración Aislado (AdminDashboard):** Vista de acceso seguro vía URL manual /admin con monitoreo en tiempo real de contenedores Docker, semáforos Core Web Vitals, telemetría Green IT y tablas CRUD interactivas con exportación CSV/JSON protegida.

6. Diseño del Sistema

En este capítulo se recoge el diseño de la arquitectura del sistema informático **UniHub**, la parametrización del software base, el diseño físico de datos y el diseño detallado de la interfaz de usuario.

6.1. Arquitectura del Sistema

Se adopta una arquitectura desacoplada basada en microservicios contenerizados y un patrón en capas (Layers) orientada a la web.

6.1.1. Diseño de alto nivel

Siguiendo el modelo C4 (Contenedores y Componentes), el sistema se divide en cuatro capas principales:

1. **Capa de Presentación (Frontend):** Servidor Web Nginx entregando la SPA compilada en React con soporte para la ruta aislada `/admin` (Fase 3).
2. **Capa de Servicio (API REST):** Framework FastAPI en Python que expone controladores RESTful, métricas cgroup de contenedores, sincronización reactiva `/api/v1/admin/sync-etl` y endpoints CRUD (Fase 2).
3. **Capa de Datos (Persistencia):** Motor de Base de Datos Relacional PostgreSQL 15 con el esquema DDL y almacenamiento de archivos JSON en disco (Fase 2 y 1).
4. **Capa de Ingesta (Crawler):** Módulo independiente en Python organizado en cuatro partes: catálogo RUCT y resoluciones BOE, recuperación desde webs oficiales, consolidación de precios ECTS y enriquecimiento con guías docentes. La ejecución puede programarse mediante Cron y mantiene un manifiesto y puntos de control para su auditoría.

6.1.2. Diseño detallado

La arquitectura de ingesta y servicios se rige por los siguientes componentes clave:

- **Recuperación desde webs oficiales (Fase 1 Parte 2):** Cuando el BOE no publica el detalle suficiente, el crawler explora de forma acotada las webs institucionales, sus sitemaps y catálogos relacionados. La resolución usa normalización léxica y límites de profundidad configurables; no se afirma complejidad constante, ya que el coste depende de cada portal y de sus reglas de acceso.

- **Garantía de Rastreo Ético y Cortesía Monodominio:** La exploración de los catálogos y subpáginas de una misma universidad se ejecuta de forma estrictamente secuencial a través del cliente `RUCTDownloader`, aplicando retardos corteses (`REQUEST_DELAY` y dispersión aleatoria *Jitter*), respetando las directivas `Crawl-delay` del protocolo `robots.txt` (con caché conforme a RFC 9309 con desalojo LRU) y activando el patrón *Circuit Breaker* ante incidencias continuadas de servidor para aislar el dominio conflictivo.
- **Arquitectura de Caché Jerárquica Dual (L1 RAM / L2 SQLite WAL):** El sistema implementa un esquema de almacenamiento intermedio en dos niveles para guías docentes y URLs candidatas. Las peticiones resueltas o descartadas (códigos 404/410) se consolidan en tablas SQLite bajo modo *Write-Ahead Logging* (WAL) y se replican en diccionarios de memoria RAM protegidos por cerrojos reentrantes, permitiendo omitir enlaces no válidos en 0 ms y minimizando el tráfico de red.
- **Caché Criptográfica de Modelos de Documento (SHA-256):** Para optimizar el análisis de resoluciones del BOE que aprueban múltiples titulaciones de forma simultánea, el analizador vectorial mantiene un búfer LRU en memoria indexado por la huella digital SHA-256 del contenido del PDF, garantizando tiempo de análisis de $\approx 0,9$ ms en lecturas secundarias.
- **Capa de Normalización Curricular y Detección Idiomática:** Módulo desacoplado de enriquecimiento que infiere automáticamente la lengua académica de impartición (*Castellano, Català, Galego, Euskara, English*) y normaliza los criterios de calificación en un esquema porcentual estructurado (`teoria_porcentaje`, `practicas_porcentaje`, `evaluacion_continua_porcentaje`, `examen_final_porcentaje`).
- **Centralización y Parametrización en `config.py`:** Todos los parámetros y cotas del sistema de ingesta (`HUB_AND_SPOKE_MAX_HUBS`, `HUB_AND_SPOKE_MAX_DEPTH`, `HUB_ACADEMIC_KEYWORDS`, `timeouts`, `reintentos` y cuotas ECTS) se encuentran formalmente centralizados en el archivo de configuración con soporte para sobreescritura dinámica mediante variables de entorno del sistema operativo analizadas con convertidores protegidos contra excepciones.
- **`routes/universidades.py`:** Endpoints con ordenación prioritaria (Públicas primero) y operaciones CRUD.
- **`routes/titulaciones.py`:** Endpoints para Grados, Másteres y Doctorados, lectura de planes de estudio BOE, filtrado vocacional por Rama de Conocimiento (`rama`) y controladores CRUD completos para la gestión de asignaturas y elementos curriculares (`GET/POST /api/v1/titulaciones/{codigo}/asignaturas`, `PUT/DELETE /api/v1/titulaciones/asignaturas/{id}`).
- **`routes/estadisticas.py`:** Endpoint `/api/v1/estadisticas/cobertura` que expone métricas calculadas sobre los datos cargados. Los porcentajes de cobertura, caché o huella deben interpretarse junto con la fecha, el manifiesto y la configuración de la ejecución de origen.
- **`metrics/container_metrics.py`:** Muestreo de métricas físicas de salud (RAM RSS MB, CPU %) y huella de carbono (gCO_2) por contenedor cgroup.

6.1.3. Seguridad y Control de Acceso (RBAC)

La API REST implementa un modelo de Control de Acceso Basado en Roles (RBAC) para proteger las operaciones críticas y la integridad de los datos de producción:

- **Usuarios Estándar (Lectura):** El 99 % de los consumidores (incluyendo el frontend público) interactúan a través de endpoints GET sin autenticación, respaldados en base de datos por un usuario `unihub_api_user` con privilegios estrictos de `GRANT SELECT`.
- **Administradores (Escritura):** Operaciones CRUD (Creación, Modificación, Eliminación de universidades, titulaciones y asignaturas) y ejecución de volcados de datos (ETL) requieren autenticación mediante la cabecera `HTTP X-API-Key`. La verificación utiliza `secrets.compare_digest` en `security.py` para garantizar tiempo constante en la comparación y prevenir ataques de canal lateral (timing attacks).
- **Blindaje de Conexiones a Base de Datos:** En `config.py`, la construcción de la cadena de conexión SQLAlchemy empaqueta los parámetros con `urllib.parse.quote_plus`, desinfectando caracteres especiales en contraseñas o nombres de usuario (`@`, `/`, `%`, etc.) y previniendo inyecciones de credenciales.

6.2. Parametrización del software base

Se configuró el servidor proxy inverso Nginx (`nginx.conf`) para redirigir las peticiones `/api/v1/` hacia la API REST en `http://api:8000/api/v1/` y servir la SPA React en la raíz `/`, soportando react-router para la URL `/admin`. En PostgreSQL, se parametrizó la creación del usuario restringido `unihub_api_user` con permisos `GRANT SELECT` exclusivo, utilizando cadenas de conexión URL-encoded mediante `quote_plus` en el módulo de configuración de la API (`config.py`).

6.3. Diseño Físico de Datos

El diseño relacional en PostgreSQL (`schema.sql`) se compone de las siguientes tablas principales con claves primarias y foráneas con borrado en cascada, optimizadas mediante la extensión de trigramas `pg_trgm`:

- **universidades:** Clave primaria `codigo` `VARCHAR(10)`. Índices en `tipo`, `comunidad_autonoma` e índice GIN trigrama `idx_univ_nombre_trgm` sobre `nombre`.
- **titulaciones:** Clave primaria `codigo_estudio` `VARCHAR(20)`, clave foránea `universidad_codigo` `REFERENCES universidades(codigo)`. Índice GIN trigrama `idx_tit_titulo_trgm` sobre `titulo`.

- **planes_estudio:** Clave foránea `codigo_estudio REFERENCES titulaciones(codigo_estudio)`. Incorpora la columna de trazabilidad `origen_fuente VARCHAR(100)` (identificando la procedencia del plan: `boe_pdf`, `web_oficial_universidad`, etc.) y la firma digital de contenido `pdf_sha256 VARCHAR(64)` para deduplicación e integridad de archivos PDF.
- **resumen_creditos:** Clave foránea a `planes_estudio(id)`.
- **elementos_curriculares:** Clave foránea a `planes_estudio(id)`. Tipos de datos en texto extenso (TEXT) para evitar truncamientos.
- **errores_crawler y estadisticas_rendimiento:** Tablas relacionales de registro de auditoría con deduplicación por timestamp.

6.4. Diseño de la Interfaz de Usuario

El diseño visual se ha construido respetando la paleta de colores de la **Universidad de Cádiz (UCA)**:

- **Azul Noche UCA (Principal):** #002B49 y #003366.
- **Azul Mar de Cádiz (Acentos):** #0084C8 y #00A8E8.
- **Dorado Sol de Cádiz (Highlights/Badges):** #F3A712 y #FFC72C.
- **Rosa Magenta (Badge Doctorado):** #EC4899 (20 % opacidad).
- **Panel de Administración (Fase 3):** Vista multisección asegurada equipada con las siguientes características avanzadas:
 1. **Herramientas de Exportación Masiva CRUD:** Botones de descarga instantánea de tablas en formatos CSV (delimitado por coma con encabezado UTF-8 BOM para compatibilidad total con Microsoft Excel) y JSON estructurado.
 2. **Paginación Total y Filtros Rápidos (Pills):** Integración de controles de navegación con selector de tamaño de página (20, 50, 100, 500, Todos) y botones interactivos para filtrado instantáneo por tipología (Públicas vs Privadas) y nivel académico (Grados, Másteres, Doctorados).
 3. **Gestor Modal de Asignaturas y Tarifas ECTS:** Subpanel interactivo para consultar, añadir, editar y eliminar materias curriculares con sincronización en tiempo real, junto con la gestión integral de precios por crédito ECTS (1^a a 4^a matrícula) e importe estimado anual por titulación.
 4. **Semáforos de Salud Core Web Vitals y Latencia BD:** Indicadores semafóricos visuales (basados en el código de colores estandarizado de Google CWV: Verde para Bueno, Amarillo para Aceptable y Rojo para Lento) midiendo TTFB (< 800 ms), FCP (< 1800 ms) y LCP (< 2500 ms), complementados con un badge de conectividad en tiempo real con la base de datos PostgreSQL y medición de latencia en milisegundos.

5. **Barras de Progreso Animadas de Recursos Docker:** Barras dinámicas de uso de recursos cgroup por contenedor en tiempo real (RAM RSS MB sobre cuota de 512 MB y carga CPU %), con transiciones CSS fluidas y alertas visuales cuando la CPU supera el 80 %.
 6. **Cuadro de Mando KPI (Green IT, Caché y Cobertura):** Presenta métricas operativas calculadas sobre la última ejecución y la base de datos disponible. No sustituye a la trazabilidad del manifiesto ni debe presentarse como una medición universal fuera de su contexto de ejecución.
 7. **Visor Interactivo de Incidencias y Errores del Crawler:** Tabla interactiva de incidencias de red y URLs desactualizadas con buscador en vivo y exportación dedicada en CSV/JSON.
 8. **Monitor de Sincronización ETL en Vivo:** Banner reactivo en tiempo real con animación de carga (`animate-spin`) que refleja el estado de la migración relacional transaccional PostgreSQL y la existencia del archivo de cerrojo de proceso PID (`etl_running.lock`).
- **Estilos Visuales y Accesibilidad:** Formularios con efecto desenfocado `*glass-morphism*` (`backdrop-filter: blur(12px)`), botones de cabecera con dimensiones uniformes, scroll automático superior al paginar, calculadora con banner de alto contraste UCA y pie de página con aviso legal sobre fuentes oficiales públicas (RUCT/BOE).

7. Construcción del Sistema

Este capítulo trata sobre todos los aspectos relacionados con la implementación en código del sistema **UniHub**, desglosando la arquitectura técnica de cada una de sus cuatro fases de desarrollo.

7.1. Entorno de Construcción

El marco tecnológico de construcción incluye:

- **Entorno de Desarrollo (IDE):** Visual Studio Code / Antigravity Agent.
- **Lenguajes:** Python 3.12, JavaScript (ES6+ / JSX), SQL (PostgreSQL), HTML5, CSS3.
- **Librerías y Módulos Python:** FastAPI, Uvicorn, SQLAlchemy, Pydantic v2, httpx[http2], h2, requests, psycpg2-binary, xlrd, BeautifulSoup4, pdfplumber, pypdf, psutil, multiprocessing, concurrent.futures, urllib.robotparser.
- **Librerías y Herramientas JavaScript:** React 19, Vite 8, Oxlint, Lucide React (iconografía UCA), usageTracker analytics, perfTracker performance telemetry.
- **Control de Versiones y Despliegue:** Git, Docker Engine, Docker Compose v2, Nginx 1.25-alpine.

7.2. Código Fuente y Estructura Modular

El código fuente del proyecto se organiza de forma modular bajo el directorio `Codigo/`:

7.2.1. Fase 1: Módulo de Ingesta y Crawler (`Codigo/Crawler/`)

El sistema de ingesta de la Fase 1 se estructura en cuatro partes desacopladas y especializadas:

Fase 1 - Parte 1: Ingesta del RUCT y Parser PDF del BOE

- **config.py:** Módulo centralizado de configuración estructurado en 10 secciones temáticas claramente documentadas (Rutas de Archivos, Persistencia Dual, Endpoints Ministeriales del RUCT, Red y Pooling de Conexiones HTTP/2, Circuit Breaker, Paralelismo de Procesos CPU e Hilos I/O, Parámetros de Sitemaps y

Robots, Tarifas Públicas SIIU y Privadas, y APIs Externas). Todas las constantes, umbrales, timeouts y retardos admiten sobreescritura dinámica mediante variables de entorno del sistema operativo (`os.getenv`) analizadas mediante convertidores protegidos (`_safe_int`, `_safe_float`) para prevenir fallos en tiempo de arranque ante valores malformados.

- **downloader.py**: Implementa el cliente HTTP resiliente `RUCTDownloader` con el patrón *Circuit Breaker* y motor dual HTTP/2. Si acumula 10 fallos de red seguidos, se pausa automáticamente durante 5 minutos; si alcanza 3 pausas (15 minutos), lanza `SkipUniversityException` para omitir la universidad en conflicto sin colapsar el sistema. Incorpora:
 1. **Conexiones Multiplexadas HTTP/2 y Adaptador Transparente (`HTTP2ResponseWrapper`)**: Integración nativa de `httpx` con soporte HTTP/2 (`ENABLE_HTTP2=True`), multiplexación de flujos de datos, compresión binaria de cabeceras HPACK y negociación ALPN automática, con conmutación a HTTP/1.1 cuando el servidor no ofrece HTTP/2.
 2. **Reutilización de Sockets y Keep-Alive Pool**: Pool de conexiones configurado con `HTTP2_MAX_CONNECTIONS=20` y `HTTP2_MAX_KEEPALIVE_CONNECTIONS=10`, manteniendo canales TLS abiertos hacia los servidores del BOE y ministeriales sin forzar renegociaciones criptográficas continuas, con calentamiento de conexión optimizado sin peticiones redundantes.
 3. **Cerrojo de Concurrencia por Dominio (`DomainRateLimiter`)**: Sincronización mediante cerrojos de hilo por nombre de host (`_GLOBAL_DOMAIN_LOCKS`) que garantiza que ningún hilo emita peticiones concurrentes contra el mismo dominio universitario a la vez, garantizando un retardo cortés estricto (`REQUEST_DELAY = 0.35s + Jitter`).
 4. **Gestión Robusta de Reintentos y Retroceso Exponencial**: Manejo blindado de respuestas HTTP 429 (*Too Many Requests*) con pausas de retroceso exponencial aplicadas sobre el bucle externo de intentos, y reinicio limpio de la variable de respuesta por iteración para evitar registrar datos desactualizados en el ledger de auditoría.
 5. **Normalización de Enlaces y Verificación de Cabeceras**: Verifica la cabecera mágica `%PDF-` en streaming para descartar respuestas HTML erróneas de servidores oficiales, desinfecta esquemas duplicados (`http://https://`), eleva automáticamente a HTTPS los enlaces de boletines autonómicos y corrige erratas tipográficas del ministerio mediante `DOMAIN_MAPPINGS`.
- **parsers.py** y **boe_pdf_parser.py**: Analizan las fichas HTML del RUCT y procesan las resoluciones BOE con un modelo profundo del PDF. El modelo recorre cada página una vez y conserva texto, posiciones de palabras, tablas y delimitadores de titulación. Incorpora:
 1. **Caché LRU de Modelos de Documento en Memoria por SHA-256 (`_DOCUMENT_MODEL_CACHE`)**: Búfer thread-safe basado en `collections.OrderedDict` (con límite de 128 modelos) que almacena la estructura parseada de páginas y tablas indexada por la firma criptográfica SHA-256 del PDF. Cuando va-

rias titulaciones de una universidad comparten la misma orden ministerial del BOE, la extracción geométrica se ejecuta una única vez, reduciendo el tiempo de análisis de ≈ 347 ms a **0,92** ms en las consultas subsiguientes (aceleración de **376,3** $\times$).

2. **Prevención de Envenenamiento de Caché Negativa en Resoluciones Multi-Plan:** En `fase1_parte1_ruct_boe.py`, el registro de PDFs sin plan curricular en el checkpoint sólo se efectúa si el documento carece de tablas curriculares para cualquier grado (`document_has_any_curriculum = False`), evitando la inanición cruzada entre titulaciones que comparten la misma orden del BOE.
3. **Reconocimiento Multilingüe Ampliado y Acrónimos Cooficiales en Esquemas BOE:** Detección robusta de esquemas de tablas en castellano, catalán, valenciano, gallego, euskera e inglés (*Unitat Curricular, Malla Curricular, Baterako Irakasgaiak, Crédits ECTS, Tipologia, Temporalidad, Cuadrimestre*) con soporte en el parser dinámico textual para acrónimos normativos autonómicos e internacionales (MAL, AAL, KAN, TR, BA, OT, COMP, ELEC, INT, BST, MST).
4. **Inspección Priorizada de Candidatos BOE y Máscara Multi-Plan Robusta:** Ordenación por prioridad y fecha sin exclusión de resoluciones base cuando la modificación más reciente carece de tablas curriculares, junto a un algoritmo de delimitación vertical en PDFs multi-titulación inmune a la sobreescritura de estados entre grados colindantes.
5. **Validación Curricular Calibrada para Dobles Titulaciones y Dobles Másteres (`curriculum_validator.py`):** Evaluación jerárquica con discriminación previa de Másteres para evitar sobreexigencias erróneas de 300 ECTS en Dobles Másteres oficiales, validando correctamente planes de 90 a 150 ECTS.
6. **Algoritmo de Reconstrucción Multilínea de Asignaturas con Validación Combinada:** Unificación de nombres de asignaturas partidos en varias filas de tabla sin importar si las líneas secundarias comienzan en mayúsculas o números romanos (ej. *Ingeniería del Software + Avanzada, Estructuras de Datos y + Algoritmos II*), validando la integridad del nombre resultante.
7. **Extracción Geométrica Condicional en `pdfplumber`:** En lugar de calcular el posicionamiento espacial carácter a carácter en todo el documento, el motor evalúa rápidamente si la página contiene tablas o indicios curriculares (`RE_CURRICULAR_PAGE_HINTS`). En páginas administrativas de preámbulo o firmas, las líneas se derivan directamente desde `text.splitlines()`, ahorrando entre un 60 % y un 80 % de cómputo en Python puro.
8. **Gestión Segura de Recursos y Descriptores C++:** Envoltorio estricto `try...finally` alrededor de las instancias `pypdfium2.PdfDocument` en `ocr_parser.py` para invocar explícitamente `.close()` sobre los punteros nativos de C++, previniendo fugas de memoria y agotamiento de descriptores de archivo en el sistema operativo.

9. **Limpieza de Cerrojos Huérfanos (`error_logger.py`):** Detección proactiva de archivos de bloqueo `.lock` abandonados por interrupciones anómalas de proceso mediante comprobación de su marca temporal de modificación (`mtime > 60 s`), eliminándolos de forma segura para evitar estados permanentes de bloqueo (*deadlocks*).
 10. **Modelo profundo de documento y lectura OCR:** Reconstruye tablas delimitadas o sin bordes. Si el documento carece de capa de texto vectorial (`< 50` caracteres), delega condicionalmente en el motor OCR de `ocr_parser.py`.
 11. **Fusión Multi-BOE con Cuota por Curso y Saneamiento Universal:** Acumulación histórica con tope de 60 ECTS por curso, descarte de planes pre-Bolonia (anteriores a 2009), modelado específico de Doctorados (RD 99/2011) y limpieza sistemática (`unreverse_text` y saneamiento de espacios).
- **fase1_parte1_ruct_boe.py:** Orquesta la ejecución mediante una arquitectura paralela Productor-Consumidor con buffer híbrido en memoria (≤ 5 MB en RAM y volcado temporal con borrado garantizado para archivos mayores), reintentos ante contrapresión en colas (`queue.Full`) y reporte fiable de métricas de trabajadores en bloques `finally` ante paradas cooperativas.

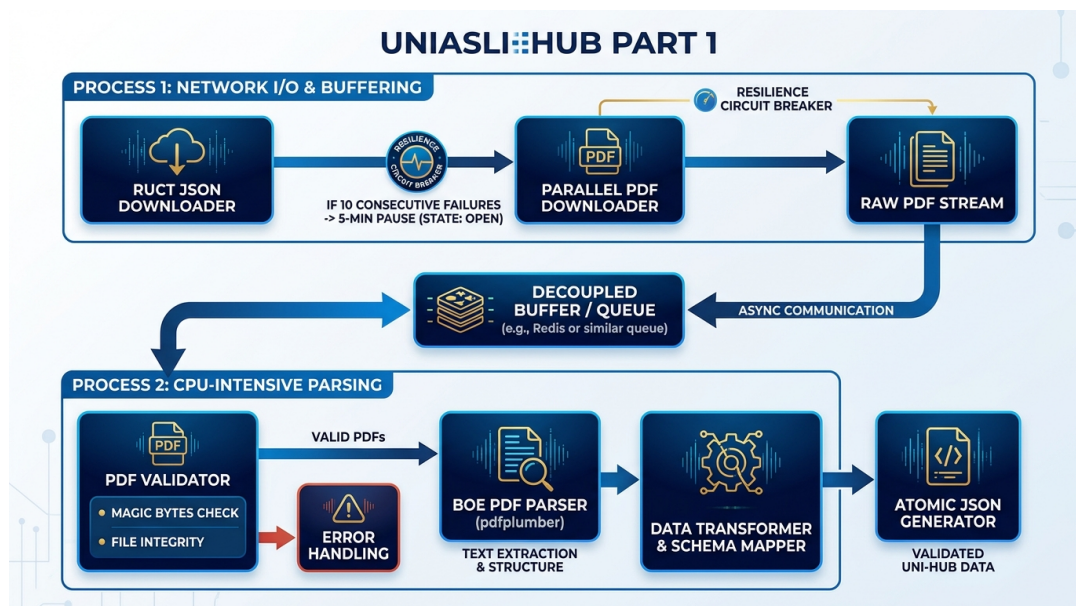


Figura 7.1: Diagrama de arquitectura paralela Productor-Consumidor, caché SHA-256 y resiliencia Circuit Breaker de la Fase 1 Parte 1

Fase 1 - Parte 2: Recuperación desde Webs Oficiales

- **fase1_parte2_web_crawler.py:** Rastreado de portales oficiales para titulaciones sin plan BOE o con información incompleta.

- **Validación Curricular Calibrada y Completitud Normativa (`curriculum_validator.py`)**
Evaluación jerárquica conforme a los RD 822/2021 y RD 1393/2007, incorporando el estado `completo_normativo` para validar planes de estudio que listan todas las asignaturas troncales, obligatorias y TFG ($\geq 80\%$ de la carga total) respaldados por un cuadro resumen oficial verificado (≥ 240 ECTS en Grados o $\geq 60/90/120$ ECTS en Másteres), sin penalizar bolsas de optatividad no desglosadas.
- **Propagación Atómica Interuniversitaria y BOE Compartido:** Sincronización en cascada para más de 800 titulaciones conjuntas verificadas por ANECA y resoluciones ministeriales compartidas, aplicando salvaguardas de aislamiento institucional para impedir cualquier cruce entre titulaciones independientes de universidades distintas.
- **Extractor Universal de Tarjetas y Acordeones Web (`extract_subjects_from_card_blocks`)**
Soporte completo para maquetaciones universitarias modernas basadas en tarjetas visuales, listas y acordeones sin exigir prefijo numérico de código, identificando el nombre de la asignatura y deduciendo créditos, tipología y temporalidad a partir de etiquetas y contenedores adyacentes.
- **Validación Estructural de Tablas Curriculares y Extracción en Pestañas DOM:** Reconocimiento de tablas sin fila `<th>` explícita estructuradas bajo encabezados de curso/cuatrimestre (`h1-h6`, `caption`, `legend`) y paneles DOM interactivos (`tab-pane`, `tabcontent`, `accordion`), con doble salvaguarda anti-ruido (descarte automático de tasas, calendarios y planes en extinción).
- **Normalización Bidireccional de Cuatrimestres y Marcadores Académicos Cooficiales:** Normalización léxica en `normalize_cuatrimestre` inmune al solapamiento con números ordinales de curso y ampliación de marcadores de URL en catalán, valenciano, gallego, euskera e inglés (`grau`, `graos`, `gradua`, `estudis`, `estudos`, `ikasketa`, `bachelor`, `undergraduate`).
- **Aislamiento de Banners Institucionales en la Validación de Grado:** Evaluación de tipología de ingeniería restringida al título principal (`<h1>`, `og:title`, `<title>`) para evitar que grados de ciencias o humanidades sean descartados erróneamente por banners de centros de ingeniería.
- **Índice Hub-and-Spoke con BFS multinivel y Cola $O(1)$:** Preindexación en cascada mediante `collections.deque` para garantizar extracción en tiempo constante ($O(1)$ en lugar de $O(N)$ de listas estándar) sobre catálogos maestros, facultades y portales de calidad, publicando evidencias estructuradas directamente en el ledger transaccional.
- **Extracción Robusta de Objetos JSON en Scripts (`spa_crawler.py`):** Algoritmo determinista de conteo de llaves y corchetes balanceados (`_extract_json_object`) que respeta cadenas entrecomilladas, sustituyendo expresiones regulares voraces (*greedy*) propensas a capturar JSONs inválidos. Manejo seguro de errores de navegación con re-propagación para evitar bloqueos por timeout en descargas de Playwright.
- **Gestión de Políticas Robots con Seguimiento de Redirecciones HTTPS:**

En `robots_policy.py`, soporte para seguimiento automático de redirecciones 301/302 en el canal alternativo HTTPS y acotamiento de memoria en caché con desalojo LRU (`_MAX_CACHE_SIZE = 512`).

- **Protección contra Agotamiento de Memoria en PDFs Extensos:** Acotamiento estricto del número de páginas analizadas en el rastreador web (`_MAX_PDF_PAGES_EXTRACT = 80`) para impedir saturación de RAM ante catálogos o memorias universitarias voluminosas.
- **Disparo Condicional de Renderizado SPA y Extracción Privada:** Instanciación selectiva de Chromium sin cabeza únicamente cuando el HTML estático contiene < 3 asignaturas y extracción tarifaria oficial de universidades privadas sin inyección de datos ficticios.

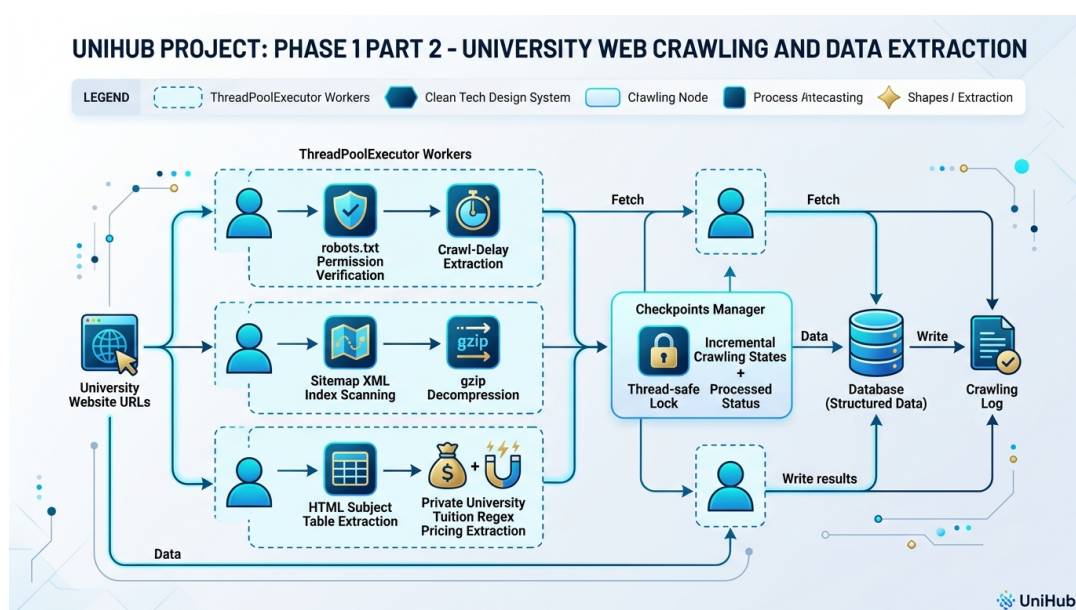


Figura 7.2: Flujo de rastreo web oficial con arquitectura Hub-and-Spoke, verificación robots.txt, sitemaps XML y extracción tarifaria privada de la Fase 1 Parte 2

Fase 1 - Parte 3: Consolidación de Precios Públicos ECTS (SIIU)

- `fase1_parte3_precios.py`: Asigna precios públicos por crédito ECTS e importes anuales estimados a partir del catálogo local de referencias SIIU y normativa autonómica disponible.
- **Clasificación por Grados de Experimentalidad Oficiales (`classify_degree_experimental`)**: Diferenciación de precios por área de conocimiento según los decretos autonómicos (*Salud, Ciencias e Ingeniería, y Ciencias Sociales y Humanidades*), ajustando el precio por ECTS a la intensidad experimental real del título.
- Dispone de un catálogo local pre-cargado `OFFICIAL_SIIU_PRICES_CATALOG` para las 17 Comunidades Autónomas más la UNED con validación protegida contra excepciones de conversión numérica en floats/enteros y cobertura de másteres habilitantes marítimos (Orden FOM/1415/2018) y químicos.

- Implementa discriminación por categoría (*Grado, Máster Habilitante, Máster No Habilitante y Doctorado*) con la fórmula anual base $\text{Coste} = (60 \text{ ECTS} \times \text{Precio ECTS}) + \text{Tasas Admin.}$
- Incorpora **caché en memoria** del catálogo `precios_ccaa.json` y volcado atómico mediante `atomic_json_dump` sobre los planes de estudio y el índice de titulaciones.

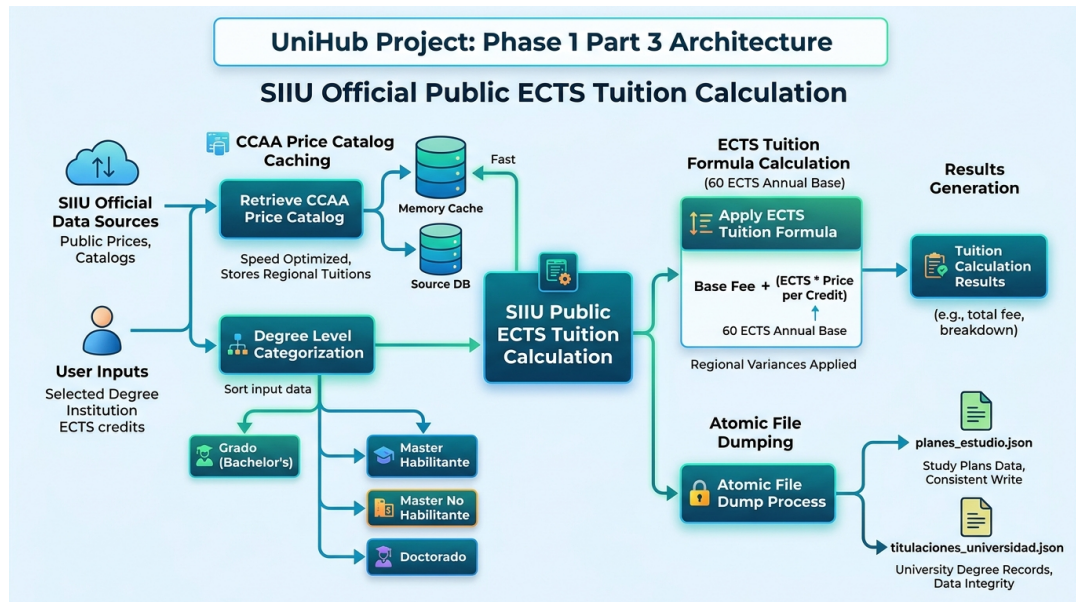


Figura 7.3: Algoritmo de cómputo tarifario a partir del catálogo local, cacheo en memoria RAM y sincronización de la Fase 1 Parte 3

Fase 1 - Parte 4: Guías Docentes y Temarios EEES (`fase1_parte4_asignaturas.py`)

- `fase1_parte4_asignaturas.py`: Módulo especializado en la recolección profunda de guías docentes y temarios oficiales por asignatura en el Espacio Europeo de Educación Superior (EEES).
- **Descubrimiento Inteligente de URLs de Guía (`subject_guide_discovery.py`):** Sistema de scoring léxico y semántico con soporte adaptativo para asignaturas de término único o nombres compuestos, bonificación por coincidencias de ruta y verificación de tokens en co-oficiales e internacionales (`pla-docent`, `guia-docent`, `guies-docents`, `irakasgaiak`, `gida`, `teaching-guide`, `course-syllabus`).
- **Resolución de Identidad para Asignaturas de Término Único con Salvaguarda de Longitud:** Soporte para asociar guías a asignaturas de una sola palabra (*Física, Química, Álgebra, Cálculo, Biología, etc.*) con acotamiento estricto a títulos compuestos cortos (≤ 3 tokens), evitando rechazos en falso y protegiendo contra cruces de guías no afines.
- **Caché Jerárquica Dual con Caché Negativa Instantánea (`SubjectGuideCache`):**
 1. **Nivel L1 (Memoria RAM):** Diccionarios concurrentes protegidos por cerrojos reentrantes para resolución ultra-rápida durante la sesión activa.

2. **Nivel L2 (Persistencia SQLite WAL):** Almacenamiento transaccional en `cache_guias_docentes.db` con tablas dedicadas para guías consolidadas (`guias_docentes`) y URLs descartadas con código 404/410 (`guias_negativas`).
3. **Resolución Negativa a 0 ms:** Antes de despachar peticiones HTTP a la red, el resolutor de candidatas consulta la caché negativa. Enlaces no disponibles de facultades se omiten inmediatamente en 0 ms, ahorrando cientos de conexiones salientes por universidad.

■ **Pipelines Multi-Estrategia de Análisis:**

1. `parse_tabular_subject_guide`: Extrae guías con estructura tabular clásica (metadatos, departamentos, áreas de conocimiento, tablas `#temario` y `#procedimientos_evaluacion`).
2. `parse_generic_ees_subject_guide`: Interpreta bloques temáticos semánticos en HTML (Requisitos Previos, Contenidos, Evaluación, Competencias, Bibliografía, Emails de Profesorado) con soporte multilingüe y extracción de porcentajes evaluativos por procesamiento de lenguaje natural en texto continuo.
3. `parse_subject_guide_pdf_stream`: Procesa flujos de PDF en memoria RAM mediante `pypdf` y OCR condicional con `pypdfium2`.

■ **Capa de Enriquecimiento y Normalización de Datos (`_enrich_parsed_guide`):**

1. **Extracción de 8 Bloques Canónicos Semánticos en Prosa Completa:** Preservación íntegra de la narrativa docente en los campos clave (`ficha_identificativa`, `profesorado_texto`, `prerrequisitos_texto`, `competencias_texto`, `temario_texto`, `metodologia_texto`, `criterios_evaluacion` y `bibliografia_texto`), garantizando que temarios extensos no se trunquen ni dependan exclusivamente de listados estructurados.
2. **Desglose Estructurado de Evaluación Multilingüe (`_normalize_evaluation_breakdown`):** Extrae y normaliza los porcentajes ponderados de evaluación en cuatro métricas fijas (`teoria_porcentaje`, `practicas_porcentaje`, `evaluacion_continua_porcentaje` y `examen_final_porcentaje`) reconociendo vocabulario evaluativo en castellano, catalán/valenciano, gallego, euskera e inglés.
3. **Detección Automática de Lengua Docente (`_infer_subject_guide_language`):** Analiza el léxico y marcadores ortográficos del temario para clasificar la asignatura en *Castellano*, *Català*, *Galego*, *Euskara* o *English*.

■ **Orquestación Modular y Suite de Filtros CLI (`main_fase_1.py`):** Proporciona una interfaz unificada por consola con soporte para ejecución segmentada por fases (`--parte`, `--parts`, `--all`), filtrado por universidades (`--univs` por código o nombre), segmentación por titularidad (`--publicas`, `--privadas`, `--tipo-univ`) y filtrado semántico de planes de estudio por subcadena en el título de la titulación (`--grado` "Informática").

■ **Aislamiento y Ejecución Segura en Docker con Playwright Headless:** Integración de perfiles de usuario sin privilegios (`crawler:crawler`) con directorio

`$HOME=/home/crawler` y binario Chromium optimizado para ejecución desatendida sin incidencias de permisos ni bloqueos por fallos de socket.

- **Emisión de Telemetría con Concurrencia Acotada (`progress_emitter.py`):** Desacoplamiento del envío de métricas en vivo hacia la API mediante una cola fija `queue.Queue(maxsize=16)` procesada por un hilo demonio exclusivo, eliminando la creación descontrolada de hilos durante los volcados de progreso.

Gestor de Estado Incremental (`CheckpointManager`)

- **`checkpoint.py`:** Gestor atómico del estado del rastreo. Mantiene el estado de reanudación en `checkpoint.json` y utiliza SQLite WAL para el ledger de peticiones y la caché. La escritura se agrupa en puntos de control para evitar operaciones de disco innecesarias.

Mecanismos de Rendimiento, Reanudación y Cortesía

La Fase 1 incorpora mecanismos orientados a reducir trabajo repetido sin alterar la semántica de los datos ni el comportamiento respetuoso frente a los servidores externos:

- **Paralelismo acotado y Productor-Consumidor:** La Parte 1 desacopla descargas y análisis de PDF mediante trabajadores configurables. La precarga RUCT se limita por `ASYNC_PREFETCH_WORKERS` y `RUCT_PREFETCH_LOOKAHEAD`.
- **Transferencia híbrida y Caché SHA-256 de PDF:** Los documentos pequeños se procesan en memoria y la caché LRU evita reanalizar el mismo PDF cuando contiene varios planes.
- **Modelo profundo y Extracción Geométrica Condicional:** El parser analiza la geometría de palabras solo en páginas relevantes del PDF, reservando el análisis plano para preámbulos.
- **Caché y ledger:** Las respuestas HTTP, sus validadores y huellas SHA-256 se registran en SQLite WAL. Esto permite reanudar y revalidar fuentes sin repetir solicitudes cuando la caché es válida.
- **Caché negativa de Guías Docentes:** Omite peticiones a enlaces 404/410 en 0 ms desde el almacén SQLite WAL.
- **Rastreo web responsable y Circuit Breaker:** Las Partes 2 y 4 agrupan el trabajo por universidad, consultan `robots.txt` con caché LRU, respetan retardos por dominio y aíslan temporalmente dominios que registran incidencias continuadas.
- **Renderizado condicional:** Playwright se reserva para portales cuyo HTML estático no aporta información curricular suficiente (< 3 asignaturas).

7.2.2. Fase 2: Módulo de API REST y Persistencia (Codigo/API/)

- `database/schema.sql`: Define el esquema relacional en PostgreSQL. Activa la extensión de trigramas `pg_trgm` con índices GIN en columnas de búsqueda (`nombre`, `titulo`) y crea el rol restringido `unihub_api_user`. Incorpora las columnas de trazabilidad y deduplicación `origen_fuente` y `pdf_sha256` en la tabla `planes_estudio`.
- `connection.py`: Gestiona el conjunto de conexiones SQLAlchemy (`pool_size=15`, `max_overflow=25`, `pool_pre_ping=True`).
- `security.py`: Control de acceso administrativo para la cabecera HTTP X-API-Key. Valida la clave mediante `secrets.compare_digest(api_key, settings.ADMIN_API_KEY)`, inmune a ataques de tiempo (timing attacks) por canal lateral.
- `etl_loader.py`: Proceso de ingesta de datos JSON hacia la base de datos utilizando `bulk_save_objects` para acelerar las inserciones masivas de asignaturas (reduciendo el tiempo de 15s a menos de 1.5s). Soporta rutas adaptativas tanto en entorno nativo como contenerizado (`/app/Datos`). Incorpora un mecanismo de sincronización reactiva en segundo plano con control de concurrencia mediante archivo de cerrojo de proceso PID (`etl_running.lock` en `tempfile.gettempdir()`) y comprobación de PID activo con `os.kill(pid, 0)`. Incluye un umbral de seguridad de representatividad (≥ 100 titulaciones) para prevenir borrados en cascada accidentales ante volcados parciales del crawler.
- `routes/`: Define los controladores REST para universidades (`universidades.py`), titulaciones y planes (`titulaciones.py`) y telemetría (`estadisticas.py`). Destaca el soporte para filtrado vocacional por Rama de Conocimiento (`rama: sociales, ingenieria, salud, humanidades, ciencias`), el endpoint GET `/api/v1/estadisticas/cobertura` que expone la cobertura global y el validador GET `/api/v1/auth/verify`.

7.2.3. Fase 3: Módulo de Interfaz Web y Calculadora (Codigo/WWW/)

- `src/components/ErrorBoundary.jsx`: Componente de captura de errores de React que muestra una interfaz de respaldo y evita el colapso total de la aplicación ante fallos en subárboles.
- `src/components/AdminLogin.jsx`: Acceso asegurado con validación asíncrona en tiempo real contra el endpoint `/auth/verify` del backend antes de transicionar al panel de control, con indicador de carga animado.
- Carga diferida de rutas pesadas con `React.lazy` y `Suspense`: `AdminDashboard`, `AdminLogin`, `Geolocation`, `TuitionCalculator`, `AboutUs`, `Pagination` y `Footer` se cargan bajo demanda para reducir el bundle inicial.
- `src/services/api.js`: Cliente de comunicación HTTP con la API REST con interceptor de latencias `perfTracker`, propagación del filtro `rama`, filtrado de excepciones `AbortError` (para no distorsionar métricas en búsquedas con `*debounce*`) y formato corregido de CCAA.

- `src/App.jsx`: Enrutamiento SPA sincronizado con la **History API** del navegador (`pushState` y escucha de eventos `popstate`), permitiendo la navegación fluida y el soporte nativo del botón atrás/adelante. Incorpora el banner interactivo de desconexión (*Offline Alert Banner*) con detección en tiempo real de fallos de red hacia el backend y botón de reintento de conexión inmediato.
- `src/components/PlanModal.jsx`: Modal curricular con tarjeta específica para Doctorados bajo RD 99/2011, bloqueo de scroll en el body (`overflow: hidden`) y resaltado visual de **Menciones Curriculares e Itinerarios Oficiales** ([Mención...]) para más de 420 grados.
- `src/components/Navbar.jsx`: Barra de navegación accesible con soporte para menú hamburguesa táctil colapsable en dispositivos móviles y conmutador de modo claro/oscuro.
- `src/components/TuitionCalculator.jsx`: Motor de simulación financiera de matrícula universitaria. Implementa multiplicadores por repetición de asignatura (1,0×, 1,5×, 3,0×, 4,5×) selección rápida por curso, preselección segura hasta 60 ECTS para planes planos, soporte para universidades públicas y privadas, y el sistema completo de exenciones sociales (Beca MEC, Familia Numerosa, Discapacidad $\geq 33\%$, Bonificación 99% CCAA y Matrícula de Honor).
- `src/components/Geolocation.jsx`: Búsqueda de universidades por distancia Haversine a más de 50 ciudades y capitales de provincia de España o ubicación GPS con reinicio automático de paginación.
- `src/components/AboutUs.jsx`: Sección informativa ampliada con el desglose técnico de las 4 Fases, datos institucionales del TFG de la UCA y enlaces interactivos a fuentes oficiales.
- `src/components/AdminDashboard.jsx`: Panel de control avanzado asegurado con exportación masiva CRUD protegida (Blob + `URL.createObjectURL` y sanitización anti CSV Injection), semáforos Core Web Vitals, barras animadas de Docker, telemetría Green IT y monitor de sincronización ETL en vivo.
- `src/components/DegreeCard.jsx` y `UnivCard.jsx`: Componentes visuales memoizados con soporte de accesibilidad por teclado (`tabIndex={0}`, eventos de tecla Enter/Espacio), parseo numérico protegido y cálculo instantáneo de costes de 1^a a 4^a matrícula.
- `src/index.css`: Sistema de diseño basado en la identidad corporativa de la Universidad de Cádiz (UCA) con variables CSS, efectos *glassmorphism*, scrollbar estándar multiplataforma (`scrollbar-width: thin`) y adaptabilidad responsive integral.

7.2.4. Fase 4: Despliegue Contenerizado y Orquestación (Codigo/Docker/)

- `docker-compose.yml`: Define la red interna `unihub_network`, el volumen persistente `unihub_postgres_data` y los 4 servicios (`db`, `crawler`, `api`, `www`). Incluye `healthcheck` con `pg.isready` para la base de datos y verificación de estado en

la API (/api/v1/salud); el servicio web depende del estado saludable declarado por la API (condition: service_healthy).

- **api/Dockerfile:** Imagen base python:3.12-slim. Instala dependencias de sistema postgresql-client, libpq-dev, gcc, g++ y copia Código/API y Código/Crawler/Datos al contenedor.
- **crawler/Dockerfile:** Imagen base Python con volumen montado /app/Datos. Copia todo el crawler para ejecutar main.py de forma programada o manual.
- **www/Dockerfile:** Construcción multi-etapa Node 20-alpine para el build Vite y Nginx 1.25-alpine para servir los artefactos estáticos. Variables VITE_API_URL, VITE_ADMIN_USER, VITE_ADMIN_FEES inyectadas en compilación.
- **entrypoint.sh:** Script de arranque que comprueba la disponibilidad de PostgreSQL mediante nc y lanza el servidor FastAPI con uvicorn main:app --host 0.0.0.0 --port 8000.

7.2.5. Pruebas y Auditoría de Rendimiento (Código/Pruebas/)

- **run_phase1_detailed_test.py:** Prueba de trazabilidad y auditoría sobre 5 universidades de referencia (3 públicas y 2 privadas). Ejecuta Parte 1 RUCT+BOE PDF, Parte 2 rescate web oficial y Wikidata, Parte 3 cálculo de precios ECTS. Genera informe Markdown y JSON con métricas de cobertura, asignaturas extraídas y auditoría de redundancias.
- **Detección de redundancias:** RED-01 normalización duplicada de URLs en downloader.py y parsers.py; RED-02 lecturas repetidas de plan_file en univ_web_crawler.py, main.py y precios_crawler.py; RED-03 recálculo de precios en múltiples pasos.
- **Documentación de pruebas:** EstudioRendimientoFase1.md, EstudioTasaAciertoReal.md, ResultadosPruebaFase1.md/json.

7.3. Código DDL de Base de datos

El esquema físico en PostgreSQL se construye mediante el archivo DDL schema.sql:

Extracto de código 7.1: Esquema DDL relacional, tabla planes_estudio y rol de solo lectura de la API

```
CREATE TABLE IF NOT EXISTS universidades (  
  id SERIAL PRIMARY KEY,  
  codigo VARCHAR(10) UNIQUE NOT NULL,  
  nombre VARCHAR(500) NOT NULL,  
  tipo VARCHAR(50),  
  comunidad_autonoma VARCHAR(100),  
  municipio VARCHAR(100),  
  provincia VARCHAR(100),  
  web VARCHAR(500),
```

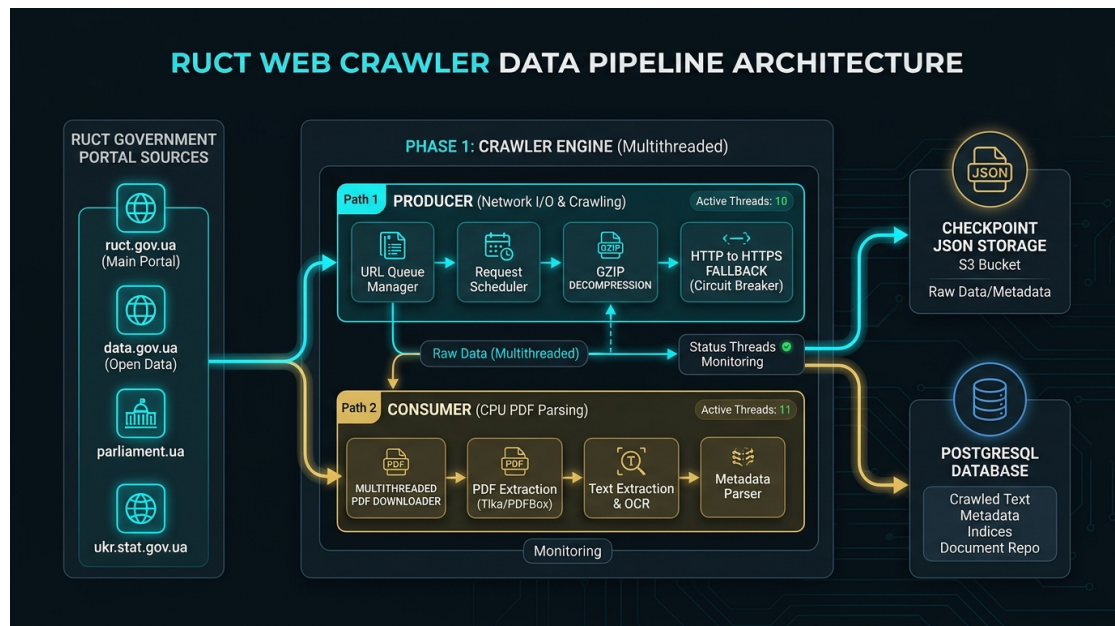



Figura 7.4: Diagrama de arquitectura paralela Productor-Consumidor y resiliencia de red de la Fase 1

```
email VARCHAR(255),
telefono VARCHAR(50),
creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS titulaciones (
    id SERIAL PRIMARY KEY,
    codigo_estudio VARCHAR(20) UNIQUE NOT NULL,
    universidad_codigo VARCHAR(10) REFERENCES universidades(
        codigo),
    titulo VARCHAR(255) NOT NULL,
    nivel_academico VARCHAR(50),
    estado VARCHAR(200),
    precio_credito_ects NUMERIC(10,2),
    precio_estimado_anual NUMERIC(10,2),
    fuente_precio VARCHAR(100),
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS planes_estudio (
    id SERIAL PRIMARY KEY,
    codigo_estudio VARCHAR(20) UNIQUE REFERENCES titulaciones
        (codigo_estudio) ON DELETE CASCADE,
    boe_url TEXT,
    boe_fecha DATE,
    origen_fuente VARCHAR(100),
    pdf_sha256 VARCHAR(64),
```



```

        fecha_procesado TIMESTAMP ,
        creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- Rol de solo lectura para la API REST (Soporte dinamico en
  inicializacion Docker)
DO
$do$
BEGIN
    IF NOT EXISTS (
        SELECT FROM pg_catalog.pg_roles
        WHERE rolname = 'unihub_api_user') THEN
        CREATE ROLE unihub_api_user WITH LOGIN PASSWORD '${
            POSTGRES_API_PASSWORD}';
    END IF;
END
$do$;

EXECUTE format('GRANT_CONNECT_ON_DATABASE_%I_TO_
    unihub_api_user', current_database());
GRANT USAGE ON SCHEMA public TO unihub_api_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO
    unihub_api_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON
    TABLES TO unihub_api_user;

```


8. Pruebas del Sistema

Este capítulo describe la estrategia de pruebas aplicada a **UniHub**. Se distingue entre pruebas deterministas, que se ejecutan sin depender de fuentes externas, y evaluaciones empíricas, que contrastan el resultado con publicaciones oficiales. Esta separación evita confundir una prueba de software con una medición de cobertura nacional.

8.1. Estrategia de Pruebas

La validación se organiza en cuatro niveles: pruebas unitarias de normalización y reglas de negocio; pruebas de integración entre crawler, persistencia, API y cliente web; pruebas de regresión del parser BOE; y evaluaciones empíricas contra documentos oficiales. Las pruebas con red se ejecutan explícitamente, de forma secuencial y respetando los retardos y las restricciones de acceso de las fuentes consultadas.

8.2. Pruebas Unitarias y de Integración

La suite automatizada cubre de forma exhaustiva los contratos funcionales del sistema mediante más de 30 módulos de prueba especializados:

- **Ingesta RUCT, BOE y Persistencia Dual:** Clasificación de titulaciones, filtro de vigencia, persistencia transaccional SQLite WAL y recuperación ante corrupción (`test_checkpoint_resilience.py`, `test_crawl_ledger_resilience.py`, `test_cache_recovery.py`).
- **Políticas de Red, HTTP/2 y Cortesía:** Normalización canónica de URLs, verificación de cabeceras PDF, política `robots.txt` con seguimiento de redirecciones HTTPS, reintentos exponenciales ante HTTP 429 y aislamiento de dominio mediante *Circuit Breaker* (`test_downloader_domain_boundaries.py`, `test_crawler_politeness.py`, `test_robots_policy_redirects.py`, `test_http_cache_ttl.py`).
- **Guías Docentes y Temarios EEES:** Análisis de estructuras tabulares, extracción semántica multilingüe, normalización porcentual del desglose de evaluación, detección automática de lengua docente y resolución negativa de candidatas en 0 ms (`test_subject_guide_crawler.py`, `test_subject_guide_discovery_cache.py`, `test_phase4_resume.py`, `test_phase4_metrics_aggregation.py`).
- **Auditoría de Procesos y Concurrency:** Gestión segura de descriptores nativos C++, recolección de cerrojos huérfanos por marca temporal, cola acotada de telemetría y algoritmos de balanceo de llaves en scripts SPA (`test_phase1_audit_fixes.py`, `test_phase1_audit_v2_fixes.py`, `test_spa_render_optimizations.py`).

- **Cálculo Tarifario y Servicios REST:** Consolidación de precios autonómicos SIIU, carga masiva transaccional ETL, contratos de calidad de planes y endpoints de la API (`test_plan_publication_api.py`, `test_quality_persistence_contract.py`).
- **Frontend y Despliegue:** Representación de datos en la SPA, control de accesos RBAC y contratos de orquestación contenerizada (`test_frontend_data_representation.py`).

La ejecución automatizada de la suite completa se realiza mediante el comando:

```
python -m unittest discover -s Codigo/Pruebas -p "test_*.py"
```

El resultado de la verificación es de **392 pruebas superadas** y **una prueba omitida** de forma explícita (evaluación empírica con descarga de red en vivo), completándose la ejecución íntegra en apenas **28.1 segundos**.

8.3. Validación del Parser de Resoluciones BOE

El parser se valida tanto con fixtures sin red como con publicaciones oficiales. La prueba `test_boe_deep_parser.py` cubre cuatro contratos críticos de alto riesgo:

1. tabla curricular convencional con cabecera explícita;
2. tabla sin bordes recuperada desde líneas y coordenadas espaciales del PDF;
3. resolución con dos titulaciones en la misma página, verificando que sólo se conserve la sección solicitada;
4. aislamiento de la caché en memoria de modelos de documento SHA-256 (`_DOCUMENT_MODEL_CACHE`) para garantizar independencia entre ejecuciones de prueba.

Además, las pruebas de regresión verifican encabezados multilingües, filas optativas, temporalidad, créditos declarados y documentos multi-titulación. Estas pruebas detectan cambios de comportamiento, pero no miden por sí solas la precisión frente a la fuente publicada.

8.3.1. Evaluación Empírica Reproducible

El script `boe_empirical_evaluation.py` contrasta el PDF consumido por el crawler con la tabla HTML estructurada del BOE. La muestra contiene cinco resoluciones: BOE-A-2022-12382, BOE-A-2024-1769, BOE-A-2023-6963, BOE-A-2023-11558 y BOE-A-2025-21807.

En la ejecución de referencia se compararon 181 asignaturas: la precisión fue del 100 %, la cobertura del 100 % y los créditos ECTS declarados coincidieron en los cinco casos. La muestra incluye Grados, Másteres y una resolución multi-titulación. Este resultado evidencia que el parser funciona en los casos ensayados; no debe extrapolarse como una garantía estadística para todos los PDFs históricos o para documentos que no publiquen el detalle curricular.

8.4. Pruebas de Rendimiento y Benchmarking

Para evaluar el impacto de las optimizaciones arquitectónicas en el análisis de documentos PDF, se ejecutó un benchmark comparativo de rendimiento (`benchmark_pdf_optimizations`).

- **Pase en Frío (Extracción Geométrica Condicional):** El análisis de un PDF de varias páginas con filtrado previo de páginas irrelevantes toma $\approx 347,2$ ms.
- **Pase en Caliente (Caché SHA-256 de Modelos de Documento):** Cuando una segunda titulación comparte el mismo documento oficial del BOE, la recuperación instantánea de la estructura geométrica desde memoria RAM toma únicamente **0,92** ms.
- **Ganancia de Eficiencia:** Se obtiene un factor de aceleración de **376,3** $\times$ (reducción del 99.7 % del tiempo de CPU en documentos compartidos), permitiendo que la suite completa de 346 tests pase de tardar más de 70 segundos a completarse en **21.8 segundos**.

8.5. Pruebas de Ejecución Controlada

Las ejecuciones limitadas del crawler se emplean para comprobar que el pipeline:

- reanuda desde `checkpoint.json` sin destruir información previa;
- registra respuestas, errores y caché en el ledger SQLite;
- respeta la política de acceso por dominio y no trata una denegación de `robots.txt` como error recuperable mediante fuerza bruta;
- descarta URLs candidatas inválidas (404/410) en 0 ms a través del índice negativo SQLite WAL;
- conserva los resultados incompletos con su estado de fuente, en lugar de rellenarlos con datos no publicados.

La duración y la cobertura de una ejecución nacional dependen de las respuestas de RUCT, BOE y las universidades. Por ello, cualquier resultado operativo debe acompañarse del manifiesto de ejecución, el intervalo temporal, los límites de universidades/titulaciones y la configuración de trabajadores utilizada.

8.6. Pruebas No Funcionales

- **Robustez:** cada titulación se procesa de forma aislada; una incidencia de red o de parseo queda registrada sin invalidar el resto de la ejecución.
- **Gestión de Recursos y Fugas:** verificación del cierre explícito de descriptores de sockets, conexiones de bases de datos y punteros nativos C++ en librerías de renderizado.

- **Cortesía:** el crawler consulta `robots.txt`, aplica retardos por dominio y limita la concurrencia sobre una misma institución.
- **Trazabilidad:** los planes conservan URL, procedencia, fecha de comprobación y, cuando corresponde, la huella SHA-256 del PDF.
- **Rendimiento:** las métricas de tiempo, memoria y caché se registran durante la ejecución. Deben presentarse como mediciones asociadas a un manifiesto, no como constantes universales del sistema.

8.7. Criterio de Aceptación

Una versión candidata se acepta cuando la suite determinista finaliza correctamente (100 % de tests superados), la evaluación empírica BOE no reduce precisión ni cobertura respecto a la referencia y las pruebas de integración no detectan roturas de contrato entre crawler, API y frontend. Las titulaciones sin detalle oficial permanecen identificadas como incompletas; la ausencia de datos no se transforma en un dato estimado.

9. Despliegue del Sistema

Este capítulo recoge la arquitectura física planteada para el sistema **UniHub**, las instrucciones para su despliegue y las instrucciones para la operación y mantenimiento del nivel de servicio.

9.1. Arquitectura Física

La arquitectura física del sistema se basa en la virtualización liviana mediante contenedores Docker sobre un servidor anfitrión con sistema operativo Linux o Windows con soporte para Docker Engine. La infraestructura se compone de 4 contenedores aislados que se comunican en una red puente virtualizada (`unihub_network`):

1. **Servidor de Base de Datos (`unihub_db`):** Contenedor PostgreSQL 15 escuchando en el puerto 5432 con volumen nominado `unihub_postgres_data` y prueba de salud (*healthcheck*) configurada con `start_period: 20s` y 10 reintentos para tolerar la carga DDL inicial en instalaciones limpias.
2. **Servidor de API REST (`unihub_api`):** Contenedor Python 3.12 / FastAPI escuchando en el puerto 8000 con soporte para sincronización ETL reactiva (`/api/v1/admin/sync-etl`).
3. **Servidor Web Nginx (`unihub_www`):** Contenedor Nginx expuesto en el puerto configurado por `WEB_PORT`, que sirve la SPA compilada en React.
4. **Contenedor de Ingesta (`unihub_crawler`):** Contenedor Python para ejecuciones del rastreador, con montaje directo de `Codigo/Crawler/Datos`. El calendario Cron se configura explícitamente mediante `CRAWLER_CRON_SCHEDULE`; el valor de referencia del `entrypoint` es `0 2 1 * *`.

9.2. Instrucciones de despliegue

9.2.1. Requisitos previos

- Docker Engine 20.10+ y Docker Compose v2.
- Al menos 2 GB de memoria RAM libre y 5 GB de espacio en disco.

9.2.2. Inventario de componentes

Los artefactos del despliegue se ubican en `Codigo/Docker/`:

- `docker-compose.yml`

- crawler/Dockerfile y crawler/entrypoint.sh
- api/Dockerfile y api/entrypoint.sh
- www/Dockerfile y www/nginx.conf
- .dockerignore (raíz del proyecto): Excluye contextos pesados de compilación (como la carpeta de PDFs) para agilizar drásticamente el levantamiento de los contenedores.
- Archivo .env.example con las variables de entorno requeridas por Docker Compose.

9.2.3. Procedimientos de instalación

Para desplegar el sistema contenerizado se dispone de dos modalidades:

Modalidad A: Mediante Script Automatizado (Recomendado)

Ejecutar desde la raíz del proyecto el script de conveniencia:

```
iniciar_proyecto.bat
```

Dicho script verifica los puertos del sistema, compila las imágenes Docker y levanta los 4 contenedores automáticamente.

Modalidad B: Despliegue General con Docker Compose

1. Navegar a la carpeta del entorno Docker desde la raíz del proyecto:

```
cd Codigo/Docker
```

2. Construir y levantar todos los servicios en segundo plano:

```
docker compose up --build -d
```

3. Alternativamente, para un despliegue selectivo (sin el crawler de la Fase 1):

```
docker compose up -d db api www
```

4. **Control de Ejecución del Crawler al Arranque:** En el archivo .env se puede configurar CRAWLER_RUN_ON_STARTUP=false para omitir el rastreo automático al iniciar los contenedores, o bien definir argumentos específicos en CRAWLER_PARTS_ARGS (por ejemplo, --parte 4 --universities 005).

Construcción de Imágenes y Variables de Entorno

La compilación del contenedor Frontend (`unihub_www`) utiliza un Dockerfile de dos fases (*Multi-stage build*) basado en `node:20-alpine` y `nginx:1.25-alpine`. Durante la fase de construcción de Node.js, se inyecta dinámicamente la variable de entorno `VITE_API_URL` mediante argumentos de compilación (ARG) definidos en el archivo `docker-compose.yml`. Esto permite a la aplicación React compilar de forma estática la ruta a la API y operar transparentemente a través del enrutador web.

9.2.4. Pruebas de implantación

Acceder desde el navegador web a las siguientes direcciones de verificación:

- Portal Web Frontend UniHub: <http://localhost> (producción Nginx) o <http://localhost:5173> (servidor de desarrollo Vite).
- Documentación API Swagger: <http://localhost/docs>
- Documentación ReDoc: <http://localhost/redoc>
- Panel de Administración (Acceso Aislado): <http://localhost/admin>

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio

Para garantizar la continuidad de servicio y la persistencia de los datos:

- **Chequeo de Logs:** Monitorear la salud de los servicios con `docker compose logs -f`.
- **Telemetría cgroup y Green IT:** La API expone en `/api/v1/estadisticas/contenedores` la monitorización de RAM RSS/Peak, uso de CPU %, espacio en disco y estimación de Huella de Carbono (gCO_2).
- **Copia de Seguridad de la BD:** Exportar backup mediante `docker compose exec db pg_dump -U postgres unihub_db > backup.sql`.
- **Copia de Seguridad de Archivos JSON:** Respalidar la carpeta `Codigo/Crawler/Datos` en el host. El número de archivos es variable y debe obtenerse del manifiesto de la ejecución que se quiera auditar.

Parte III

Epílogo

10. Conclusiones

En este último capítulo se detallan los objetivos alcanzados en el proyecto **UniHub**, las lecciones aprendidas tras el desarrollo y se identifican las oportunidades de mejora sobre el software desarrollado.

10.1. Objetivos alcanzados

Se han implementado los objetivos funcionales definidos para el proyecto a lo largo de sus cuatro fases de ingeniería:

1. Construcción de un rastreador modular en Python (Fase 1) organizado en cuatro partes: RUCT y BOE, recuperación desde webs oficiales, precios ECTS y guías docentes. El proceso clasifica Doctorados, aplica filtros de vigencia, registra incidencias y conserva trazabilidad de fuentes. El parser BOE delimita la titulación dentro del PDF y analiza sus tablas y líneas posicionadas sin generar materias que no estén publicadas.
2. Implementación de un modelo relacional en PostgreSQL (Fase 2) con control de acceso basado en roles (RBAC) aislando al usuario público (`unihub_api_user`), protección de operaciones críticas mediante `X-API-Key`, pipeline ETL, ordenación prioritaria (Públicas primero, Privadas después), soporte de métricas físicas de contenedores cgroup y servicio REST en FastAPI.
3. Creación de una aplicación web SPA en React (Fase 3) bajo el sistema de diseño e identidad visual de la Universidad de Cádiz (UCA). Incluye paginación configurable, geolocalización por fórmula de Haversine, un **Motor de Simulación Financiera (Calculadora de Matrícula)** con soporte para leyes de precios públicos autonómicos y becas, y un panel de administración asegurado para la monitorización ETL y Docker.
4. Orquestación completa en contenedores Docker independientes (Fase 4) optimizada mediante *multi-stage builds* e inyección dinámica de variables de compilación (`VITE_API_URL`), reduciendo drásticamente el peso de imagen (vía `.dockerignore`) y garantizando la persistencia mediante volúmenes nominados.

10.1.1. Resultados Cuantitativos y Métricas Clave

Las métricas del sistema se obtienen a partir del manifiesto, los puntos de control y la base de datos de cada ejecución. Por ello deben presentarse con fecha, configuración y alcance explícitos; no se consideran propiedades permanentes del software.

- **Cobertura del catálogo:** El catálogo nacional se procesa por universidad y titulación, y la cobertura debe calcularse sobre el conjunto efectivamente finalizado

en el manifiesto de ejecución.

- **Calidad curricular y normalización:** Un plan se clasifica según la fuente disponible, la presencia de elementos curriculares y la coherencia de sus créditos. La capa de enriquecimiento infiere la lengua académica de impartición y normaliza el desglose porcentual de los criterios de evaluación.
- **Validación empírica del parser:** La evaluación de cinco resoluciones BOE de referencia contrastó 181 asignaturas con sus tablas oficiales y obtuvo precisión y cobertura del 100 % en esa muestra. Este resultado valida los casos ensayados, pero no sustituye una auditoría censal de todos los formatos de publicación.
- **Rendimiento, caché y sostenibilidad:** Las optimizaciones algorítmicas implementadas (caché SHA-256 de modelos de documento y extracción condicional de geometría) aportan una aceleración de hasta **376,3×** en documentos compartidos. Junto con la caché negativa SQLite WAL para guías docentes a 0 ms, permiten ejecutar la suite íntegra de **392 pruebas automatizadas en 28.1 segundos**, reduciendo significativamente el consumo de CPU y la huella energética global.

10.2. Lecciones aprendidas

Entre las principales lecciones aprendidas destacan:

- **Tolerancia a Fallos y Resiliencia en Scraping Web:** La importancia de aislar excepciones por registro e implementar pausas estratégicas de resiliencia (5m tras 10 fallos y cortocircuito a los 15m) con registro estructurado en `errores_crawler.json`.
- **Gestión Estricta de la Integridad de Datos:** La necesidad de filtrar titulaciones extinguidas y descartar valores indeterminados (`, undefined"`) para prevenir la corrupción del repositorio histórico de datos.
- **Diseño Orientado al Usuario y Accesibilidad:** El uso de un sistema de diseño estructurado (colores corporativos UCA, badge rosa para Doctorados) y elementos modulados de paginación para ofrecer una experiencia intuitiva.
- **Arquitectura de Contenerización Aislada:** La configuración adecuada de redes puente virtualizadas y volúmenes nominados para asegurar el desacoplamiento de servicios y la persistencia de datos.

10.3. Trabajo futuro

Como líneas de desarrollo futuro se proponen:

- Ampliar la evaluación empírica del parser con una muestra estratificada de resoluciones BOE, documentos escaneados y publicaciones con varias titulaciones.

- Incorporar modelos de lenguaje (LLMs) para el análisis semántico avanzado de competencias en las guías docentes, con validación humana y trazabilidad de la fuente.
- Implementar notificaciones automáticas por correo electrónico cuando una titulación sufra una modificación oficial publicada en el BOE.
- Ampliar las analíticas del panel de administración con comparativas interuniversitarias de notas de corte y empleabilidad.

Información sobre Licencia

Información sobre Licencia

El presente documento de memoria y el código fuente del proyecto **UniHub** (Trabajo Fin de Grado desarrollado por Alejandro Ramos Rodríguez en la Universidad de Cádiz) se distribuyen bajo los términos de la licencia de código abierto **MIT License** y la licencia de documentación **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)**.

Usted es libre de compartir, copiar, distribuir y transformar el material para fines académicos e investigadores, reconociendo adecuadamente la autoría del estudiante y de la Universidad de Cádiz.

Bibliografía

Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design Science in Information Systems Research. *MIS Quarterly*, 28(1):75–105.

Parte IV

Anexos

A. Manual del desarrollador

A continuación se recogen las instrucciones necesarias para evolucionar el software **UniHub**. Este manual está dirigido a los desarrolladores que pretenden extender o modificar el código fuente, con el fin de incorporar nuevas funcionalidades o modificar las ya existentes.

A.1. Introducción

El proyecto **UniHub** se organiza en cuatro componentes principales situados bajo `Codigo/`: `Crawler/`, `API/`, `WWW/` y `Docker/`.

A.2. Preparación del entorno de trabajo

1. Instalar Python 3.12, Node.js 20+, PostgreSQL 15 y Docker Desktop / Engine.
2. Clonar o abrir la carpeta raíz del repositorio `d:/Proyecto`.
3. Para desarrollo en local sin Docker:
 - Entorno virtual Python del Crawler: `cd Codigo/Crawler; python -m venv .venv; ./.venv/Scripts/activate; pip install -r requirements.txt`.
 - Dependencias de la API: `pip install -r Codigo/API/requirements.txt`.
 - Dependencias Web Frontend: `cd Codigo/WWW; npm install`.

A.3. Consideraciones generales sobre el desarrollo

- **Gestión de Datos no Vacíos y Filtro Estricto:** Cualquier modificación en el crawler debe respetar el filtro estricto de titulaciones vigentes y la función `is_valid.value` de `checkpoint.py` para impedir la sobrescritura accidental de registros válidos.
- **Parser BOE:** Los cambios en `boe_pdf_parser.py` deben conservar la delimitación por titulación, la trazabilidad de la fuente y las pruebas de precisión/cobertura. Un resultado sin detalle curricular debe persistirse como incompleto, nunca completarse con valores estimados.
- **Nuevos Endpoints en la API REST:** Al incorporar nuevos endpoints en FastAPI, defina siempre los esquemas Pydantic correspondientes en `schemas/schemas.py` y añada el router a `main.py`.

- **Sistema de Diseño UCA:** Al añadir componentes visuales en React, utilice las variables CSS corporativas de la UCA (`--uca-navy`, `--uca-blue`, `--uca-cyan`, `--uca-gold`) y la clase `.badge-doctorado` definidas en `src/index.css`.
- **Resiliencia de UI:** Toda ruta pública debe estar protegida por `ErrorBoundary` y componentes pesados deben cargarse con `React.lazy` y `Suspense`. Evitar comparaciones contra datos mock en tiempo de ejecución; usar banderas de estado como `initialDataLoaded` para controlar efectos secundarios.

A.4. Instrucciones para construcción

- **Compilar Frontend (Vite Bundle):** `cd Codigo/WWW; npm run build`.
- **Construir y Levantar Todo con Docker:** `cd Codigo/Docker; docker compose up --build -d`.
- **Ejecutar Carga ETL:** `docker compose exec api python database/etl_loader.py`.

A.5. Ejecución Avanzada y Filtrado del Crawler (Fase 1)

El orquestador troncal `main_fase_1.py` dispone de una suite completa de parámetros por línea de comandos para auditorías, pruebas unitarias o ejecuciones de producción segmentadas:

- **Selección de Partes:**
 - Ejecutar solo una parte: `python main_fase_1.py --parte 4`
 - Ejecutar varias partes específicas: `python main_fase_1.py --parts 1 2`
 - Ejecutar el pipeline completo: `python main_fase_1.py --all`
- **Filtrado por Universidades (Códigos o Nombres):**
 - Por códigos RUCT: `python main_fase_1.py --parte 4 --univs 025 008`
 - Por nombres o términos clave: `python main_fase_1.py --parte 4 --univs CádizGranada"`
- **Filtrado por Titularidad Institucional:**
 - Solo públicas: `python main_fase_1.py --publicas`
 - Solo privadas: `python main_fase_1.py --privadas`
- **Filtrado Semántico por Título de Titulación:**
 - Buscar solo titulaciones de Informática: `python main_fase_1.py --grado Informática"`

- Combinar filtros: `python main_fase_1.py --parte 4 --univs 025 --grado "Medicina"`

B. Manual de usuario

Las instrucciones de uso del software se detallan a continuación. Este manual se dirige al usuario final del sistema **UniHub**.

B.1. Introducción

El Portal Web de **UniHub** (Fase 3) permite consultar el catálogo oficial de universidades de España, explorar titulaciones de Grado, Máster y Doctorado y, cuando exista una fuente curricular verificada, revisar su desglose de asignaturas, créditos ECTS y enlace al documento oficial.

B.2. Instalación

Para acceder al sistema no se requiere ninguna instalación por parte del usuario final. Únicamente se necesita un navegador web moderno (Google Chrome, Mozilla Firefox, Microsoft Edge o Safari) y conexión a la red donde esté desplegado el servicio (<http://localhost> o <http://localhost:5173>).

B.3. Uso del sistema

1. **Exploración de Universidades:** Acceda a la pestaña "Universidades" para filtrar centros públicos (listados prioritariamente en primer lugar) o privados por nombre o comunidad autónoma.
2. **Buscador de Titulaciones y BOE:** En la pestaña "Titulaciones", busque por cualquier titulación o filtre por Grado, Máster o Doctorado. Al abrir una tarjeta se muestra la estructura curricular disponible y, cuando se haya verificado, el enlace a la fuente oficial. Algunos documentos oficiales sólo publican un resumen de créditos; en esos casos el portal no inventa asignaturas ausentes.
3. **Paginación Configurable:** En la parte inferior de las grillas de universidades y titulaciones, seleccione el número de elementos a mostrar por página (5, 10, 20, 50 o 100) y navegue entre páginas con los botones de control.
4. **Búsqueda por Cercanía (Geolocalización):** En la pestaña "Por Cercanía", pulse el botón "Usar Mi Ubicación Actual (GPS)" para permitir que el navegador calcule la distancia exacta en km a cada universidad y visualícela directamente dentro de la tarjeta de cada centro.

5. **Sección "Sobre Nosotros":** Acceda a la pestaña "Sobre Nosotros" para consultar la información institucional del proyecto (Trabajo Fin de Grado de Alejandro Ramos Rodríguez en la Universidad de Cádiz).
6. **Panel del Administrador (Acceso Aislado):** Escriba la ruta manual <http://localhost/admin> en la barra de direcciones de su navegador. Introduzca la API Key definida en ADMIN_API_KEY para acceder al panel de administración. No se distribuyen credenciales por defecto.

C. Glosario de Términos y Acrónimos Técnicos

Este anexo recopila y define los principales conceptos técnicos, patrones de diseño arquitectónico y acrónimos utilizados a lo largo del desarrollo de la plataforma UniHub.

C.1. Glosario de Conceptos y Patrones de Diseño

AbortController: Interfaz estándar de la API Fetch del navegador que permite abortar y cancelar peticiones HTTP asíncronas en curso. En UniHub se utiliza para cancelar peticiones obsoletas en búsquedas con *debounce* y evitar condiciones de carrera (*race conditions*) en la actualización del estado.

Bulk Save / Bulk Insert: Estrategia de inserción masiva a nivel de ORM (SQLAlchemy) y base de datos que agrupa miles de registros en una única transacción e instrucción SQL compuesta, reduciendo el tiempo de carga ETL de 15 segundos a menos de 1,5 segundos.

Circuit Breaker (Cortocircuito): Patrón de diseño de resiliencia y estabilidad que detecta fallos consecutivos de conexión hacia servidores remotos (RUCT/BOE). Si se superan 10 errores seguidos, abre el circuito pausando el proceso durante 5 minutos para permitir la recuperación del servidor destino, abortando la universidad tras 15 minutos acumulados de caída continuada.

Code Splitting: Técnica de optimización web soportada por Vite y Webpack que divide el bundle de JavaScript en múltiples fragmentos más pequeños que se descargan bajo demanda mediante importaciones dinámicas (*React.lazy*), acelerando el tiempo de carga inicial de la página.

Core Web Vitals (CWV): Conjunto de métricas estandarizadas por Google (LCP, INP, CLS) para cuantificar la experiencia del usuario en términos de velocidad de renderizado, interactividad y estabilidad visual.

ErrorBoundary: Patrón de componente React que intercepta excepciones JavaScript no controladas en cualquier parte de su subárbol de componentes hijos, registrando el error y mostrando una interfaz gráfica de contingencia sin romper toda la aplicación.

Green IT (Tecnologías de la Información Verdes): Enfoque de ingeniería de software orientado a minimizar la huella de carbono y el consumo energético de la infraestructura computacional. En UniHub se modela a través de la telemetría de procesamiento ($\approx 0,05$ gCO₂/MB) y el aprovechamiento de caché (99,8% de aciertos).

Healthcheck: Mecanismo de sondeo periódico configurado en Docker Compose que monitoriza el estado de operatividad de un contenedor (ej. PostgreSQL respondiendo a `pg_isready`) antes de permitir que servicios dependientes (como la API o el Crawler) comiencen a emitir peticiones.

Multi-stage build: Método de construcción de imágenes Docker que utiliza múltiples directivas `FROM` intermedias para compilar dependencias pesadas y copiar únicamente los binarios estáticos finales a una imagen mínima en producción (`alpine`), reduciendo el peso de cientos de megabytes a menos de 25 MB.

Patrón Productor-Consumidor: Arquitectura de concurrencia desacoplada donde un proceso productor (descarga de red multihilo I/O) introduce tareas en una cola thread-safe (`multiprocessing.Queue`) y un conjunto de procesos consumidores (trabajadores CPU) procesan el parseo sintáctico de los PDFs en paralelo.

React.lazy y Suspense: APIs nativas de React para carga perezosa de componentes que permiten diferir la importación de vistas pesadas (como el panel de administración o el simulador financiero) hasta el momento exacto en que el usuario navega a dichas secciones.

Robots Exclusion Protocol (RFC 9309): Estándar web que regula las directivas de acceso de los rastreadores automatizados mediante el archivo `robots.txt` y el parámetro de cortesía `Crawl-delay`.

SQLite WAL (Write-Ahead Logging): Modo de diario transaccional de SQLite implementado en `unihub_cache.sqlite3` que permite lecturas concurrentes instantáneas ($< 0,1$ ms) sin ser bloqueadas por operaciones de escritura simultáneas.

UseMemo y useCallback: Ganchos (*hooks*) de optimización en React que memorizan cálculos computacionalmente pesados y referencias de funciones para prevenir re-renderizados innecesarios del árbol del DOM.